Learning Handbook: From Zero to nsF5 Steganography

nsF5 Steganography Project

2026 年 9 月 8 日

目录

Introduction - How to Use This Handbook

This is a handbook that teaches through code. It does not replace a textbook or the project thesis. Instead, it arranges the information-hiding and machine-learning ideas behind F:\Steganography in an order a beginner can actually follow: intuition and examples first, then the project implementation, then hands-on experiments.

0.1 Who This Is For

Assumed starting point - the handbook fills every other gap for you:

- Comfort with a Windows PC and installing software;
- Some college math exposure, or willingness to skip a formula and return later;
- A little programming is helpful but not required - Chapter 2 brings Python up to working level;
- At least 12 weeks of 6-8 hours per week; for a deep 6-12 month track, see Appendix F.

Tip

If you have never installed Python, do not worry: the first two weeks are designed exactly for that situation. The goal is not to become a programmer but to become someone who can read and modify this project.

0.2 Reading Conventions

Marker	Meaning
Tip	Plain-language explanation or analogy that removes a common stumbling block
Watch out	A trap beginners frequently hit: types, mismatched parameters, misread statistics
Try it	A hands-on experiment you must run; reading alone does not count as learning

Think about it	An open question with no single right answer, used to test real understanding
Read the code	Open a specific file and map the concept onto the implementation

0.3 The 12-Week Quick-Start Roadmap (full 6-12 month track in Appendix F)

The route follows a natural order: carrier basics -> steganography -> steganalysis -> machine learning -> engineering -> capstone. Every week maps to runnable code or a reproducible experiment:

Week	**Topic**	**Chapter**	**Hands-on outcome**
1	Digital images and binary	Ch. 1	Inspect pixels and bit planes yourself
2	Python / NumPy / Pillow	Ch. 2	Run the GUI or run_e2e.py; read/write image arrays
3-4	LSB hiding and blind steganalysis	Ch. 3	Hide a message, then detect it with chi-square/RS
5	Matrix embedding and F5	Ch. 4	Use Hamming codes to change less and hide more
6	nsF5, wet paper, hash keying	Ch. 5-6	Explain the wet-paper solver in ns5_core.py
7	Machine-learning foundations	Ch. 7	Explain features, labels, overfitting, AUC
8-9	ML-based steganalysis	Ch. 8	Run the v1 and v2 pipelines; explain SRM and dual models
10	C++ / GPU / data engineering	Ch. 9	Understand acceleration, self-checks, multi-source data
11-12	Capstone and presentation	Ch. 10-11	Complete and present a small improvement experiment

Try it

Copy this table into a weekly checklist (Appendix C has a ready-made one). Tick a row every weekend and answer that chapter's "Think about it" question.

Want to go deeper and produce results?

The above is the 12-week "get started" route; if you have 6-12 months, see **Appendix F's in-depth roadmap** - the same content, but slowed and deepened over "Foundation / Intermediate / Advanced / Project" phases, with an optional "algorithm / detection / engineering / research" track.

0.4 What You Will Be Able to Do

- Explain steganography, steganalysis, embedding efficiency, shrinkage, wet paper coding, and syndromes in your own words;
- Say why naive LSB hiding is exposed by chi-square and RS analysis;
- Work one p=3 Hamming embedding example by hand (at most one coefficient changed per block);
- Explain the real difference between F5 and nsF5, and why eliminating shrinkage matters;
- Explain how image-hash keying gives decoder self-synchronization, and where its limits are;
- Explain the full supervised-learning pipeline, including why photo-grouped cross-validation is essential;
- Understand the v1 11-D features and the v2 143-D features, including SRM statistics and the 143d/53d dual-model strategy;
- Explain what C++ DLLs and GPU code accelerate and why bit-level consistency checks matter;
- Complete a small, honest improvement experiment and write up the results.

0.5 Project Map: Know the Code Before You Study It

The project is a research tool: it runs algorithm experiments and ships as a GUI application. Remember these files first; every chapter returns to them:

File	Role	Chapter
src/image_io.py	Image I/O wrappers for 8-bit grayscale and color images	Ch. 2
src/ns5_core.py	Algorithm core: Hamming codes, wet paper, hash keying, embed/extract	Ch. 3-6
src/steganalysis.py	Blind steganalysis: chi-square, RS, combined verdict	Ch. 3

src/efficiency.py	Theoretical and measured code-family / efficiency curves	Ch. 4
src/matrix_demo.py	Teaching demo of syndrome lookup	Ch. 4
src/fsfeatures.py + cpp/fsfeatures.dll	ctypes binding for the v1 11-D features	Ch. 8
src/featurize_v2.py + src/srm_filter.py	v1.4: SRM preprocessing and the 143-D v2 feature set	Ch. 8
src/make_dataset.py / train_model.py	Dataset generation (v1/v2, SRM, multi-source) and training	Ch. 8
src/ml_predict.py	Single-image ML prediction (143d/53d dual models + sensitivity)	Ch. 8
src/cppembed.py + cpp/nsf5embed.dll	C++ embed/shuffle hot path and self-checks	Ch. 9
gpu/*.py	PyTorch batched features (v1 11-D / v2 143-D) and GPU training	Ch. 9
src/gui.py	Tkinter GUI with teaching panels	Ch. 9
src/test_core.py and friends	Regression tests for embedding, analysis, false positives	All

Read the code

Keep the project window open from now on. Whenever the handbook names a file, switch to it. From Chapter 2 onward most topics assume you run before you read.

0.6 Learning Tips

- **Run first, understand later.** End-to-end scripts remove most paper confusion;
- **Digest formulas twice.** First pass: conclusion and intuition. Second pass: derivation;
- **Retell in your own words.** If you can explain a chapter summary to a friend, you own it;
- **Keep an experiment log.** Parameters and outputs are the most valuable material for your capstone;
- **Trust the tests.** test_core.py and test_steg.py are the fastest judge of whether you broke something.

Think about it

Before you start, spend five minutes answering: what is the difference between encryption and steganography? If you cannot say, that is perfect - Chapter 3 starts there.

0.7 What Changed for v1.4.0 (2026-09-06)

The first draft of this handbook tracked project v1.3. On September 6 the project moved to v1.4.0, and this edition is synchronized with it. Chapters 1-6 (the steganography algorithms) are unchanged; the machine-learning and engineering chapters now include sections 8.8 and 9.7, and older scores are labeled as ”v1 baselines”.

- ML detection grew from 11-D features with LR/XGB (AUC 0.75-0.79) to a 143-D LightGBM default (0.9085 average) plus a 53-D interpretable model (0.9227); see 8.8;
- New SRM high-pass preprocessing, 143-D v2 features, 12 embedding variants, real JPEG clean samples, and a full BOSSbase multi-source dataset; see 8.8 and 9.7;
- The repository moved to a new GitHub home (Yukinoshita-lin/nsf5-steganography) and the license is now Apache-2.0 with a NOTICE file;
- The glossary, command sheet, weekly checklist, and concept-to-code map were extended for v1.4.

Chapter 1 - Digital Images and Binary (Week 1)

Try it

Goals: cement the intuition that an image is a matrix of numbers; do binary and byte arithmetic by hand; explain why both the eye and statistics tolerate small pixel perturbations. We use the project's real carrier (img/cover.png) to open up an image ourselves.

1.1 A Digital Image Is Just a Pile of Numbers

Zoom an **8-bit grayscale image** down to the pixel level: it is a 2-D table of integers from 0 to 255. 0 is all black, 255 is all white, and between are shades of gray. The computer only stores the table's "size" and "numbers", not the picture your eye sees.

A **color image** is three such tables stacked: R (red), G (green), B (blue) channels. A common RGB image has 3 values (0-255) per pixel. For instance an orange pixel can be written R=255, G=128, B=0.

In code, a grayscale image is a 2-D array, and a color image is a 3-D array. The project's `src/image_io.py` wraps this into two functions: `load_as_gray()` reads any supported image into a grayscale array, and `save_image()` saves an array to PNG.

Tip

Math uses "row, column" for matrices; image processing says "height, width": a 512x512 image is 512 rows, 512 columns. The array shape `(512, 512)` is exactly "rows first".

1.2 Binary, Bytes, and Bits

Computers only know 0 and 1. A single 0 or 1 is **1 bit**; 8 bits make **1 byte**. 8 bits can represent 2 =256 combinations, exactly matching gray values 0-255.

For example, decimal 200 = 128+64+8, written in 8-bit binary as `11001000`; its **least significant bit (LSB)** is the rightmost `0`. Change 200's LSB to 1, and you get 201 - to the eye, gray 200 and 201 are nearly indistinguishable.

”Least significant bit” is one of the most important concepts in this whole handbook: LSB hiding is secretly modifying pixel LSBs to embed secret bits.

Try it

Open an interactive Python (or write a small script) and verify:

Experiment: decimal, binary, and LSB

```
v = 200
print(bin(v))              # 0b11001000
print(v & 1)               # lowest bit = 0
print(v ^ 1)               # flip lowest bit = 201
print(v & 0b11111110)      # clear lowest bit = 200
print(v & 0b11111110 | 1)# clear then set = 201
```

1.3 Why Images Have ”Room” to Hide Things

Images contain lots of **redundancy**, understood at two levels:

- **Visual redundancy**: the eye is insensitive to tiny brightness differences and high-frequency detail. Changing a pixel from 128 to 129 is nearly invisible;
- **Statistical redundancy**: natural images have highly correlated neighboring pixels, with most information concentrated in low frequencies. JPEG compression shrinks files precisely by removing this redundancy;
- **The LSB plane is near-random**: the lowest bits of a natural photo look ”like noise”, supplying a natural ”cover environment”.

So two uses of redundancy appear: **compression** wants to remove redundancy to shrink the file, while **steganography** wants to stuff secrets into redundancy while disturbing visual/statistical features as little as possible. They fight over the same ”modifiable space”, which is why many hiding algorithms are designed for specific compression formats.

Watch out

Do not think ”invisible = safe”. Invisibility is only the weakest requirement; a professional detector uses statistics, and plain LSB replacement leaves a very obvious statistical trace - exactly what Chapter 3’s chi-square and RS analysis catch.

1.4 Bit Planes: Decompose an Image into 8 Layers

After writing gray values as 8-bit binary, you can decompose the image into 8 binary images by ”which bit”, called **bit planes**. The most significant (MSB) plane determines the big outline; the LSB plane is almost all fine noise.

First, split the project's real carrier (img/cover.png) by hand:

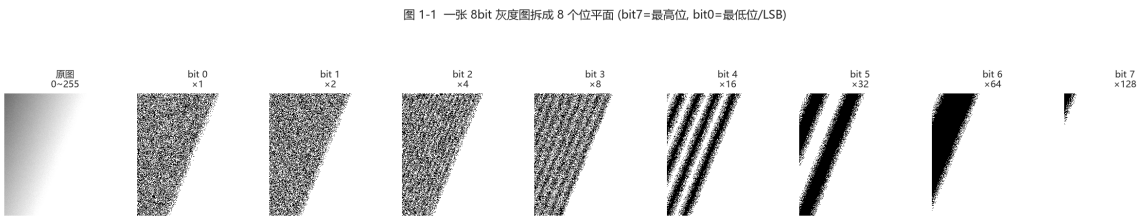

图 1: Fig. 1-1 bit-plane decomposition

Fig. 1-1 (real data: far left is the original (0-255); the right 8 cells are bit 7 (MSB, x128) down to bit 0 (LSB, x1). Bit 7 decides the outline; bit 0 is almost all fine noise)

Reading the figure:

- The higher bit planes are the more ”structured” (bit 7, 6, 5 resemble the original outline);
- The lower bit planes are the more ”noise-like” (bit 0, 1, 2 full of random specks);
- Precisely because the LSB plane is already near-random, adding pseudo-random secret bits there is hard to notice visually or by a simple histogram - but statistics can tell it was ”randomized” (Chapter 3).

Now do the ”cumulative reconstruction” by hand too, to understand ”each layer must be multiplied by its weight before adding”:

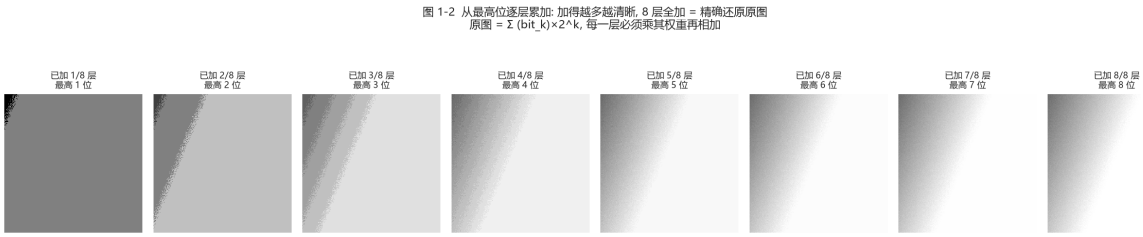

图 2: Fig. 1-2 layer-by-layer reconstruction

Fig. 1-2 (real data: accumulating from the MSB - first only bit7, a dark blocky outline; each added layer gets clearer; with all 8 layers it equals exactly the original)

This is the key sentence of ”overlaying”:

$$\text{original image} = \sum_{k=0}^{7} \text{bit plane}_k \times 2^k$$

A bit plane is only 0/1; **it must be multiplied by its weight 2^k before adding**. If you just ”OR” the 8 layers or add them ignoring weights, you do not get the original. That is why the project ”changes LSB” with `& 0xFE | bit` (clear the LSB then set it) rather than integer add/subtract.

Tip

In Fig. 1-1 you see ”high bits make the outline, low bits look like noise”. This intuition matters a lot: **changing high bits = visible to the eye; changing low bits = invisible to the eye but measurable statistically**. Hiding only touches the low bits, precisely because ”low bits already look noisy, so changing them is not obvious” - both steganography’s opportunity and detection’s breakthrough.

Try it

Open the interactive lab (`webapp/index.html`) block 1 (bit-plane lab): pull out bit 0, bit 4, bit 7 individually and compare; switch to ”cumulative reconstruction”, turn off bit 0, 1, 2 and watch the image go ”flat”, then turn all 8 on to see it exactly recover the original.

Count pixels and LSB values (numpy operations you will see everywhere)

```
import numpy as np
img = np.asarray([[200, 201], [128, 129]], dtype=np.uint8)
print(img.shape, img.dtype)   # (2, 2) uint8
print(img & 1)                # LSB plane [[0,1],[0,1]]
print(np.unpackbits(np.array([200], dtype=np.uint8)))
# [1 1 0 0 1 0 0 0]  <- MSB first
```

1.5 Back to the Project: Meet cover and stego

Two fixed terms in this field: the original carrier without a secret is **cover**, and the image after hiding is **stego**. Both appear all over: `img/cover.png` is the demo carrier, and `output/stego_*.png` are the embedded images.

Back to the code

Open step 1 of `src/run_e2e.py`: it uses numpy to generate a 256x256 smooth ”cover” and saves it to `img/cover.png`. Look closely - `np.linspace` makes the gradient background, then a little noise is added. Understanding this code shows how a natural-looking image ”grows” out of numbers. (Fig. 1-1/1-2 use exactly this image.)

1.6 Summary and Self-Check

- A grayscale image = a matrix of 0-255 integers; a color image = three channel matrices;
- The LSB is the lowest bit of a pixel's binary value; flipping it has a tiny visual effect (Fig. 1-1);
- Compression removes redundancy; steganography uses it; they are at odds;
- cover = original carrier, stego = the image with a secret;
- High bits make the outline, low bits look like noise - the root of "hiding in low bits, detection measuring low bits" (Fig. 1-2).

Think about it

Is a pure black (all 0) image a good LSB carrier? What about a pure random-noise image? Write it down, then come back after Chapter 3. (Hint: whether the LSB plane is "near-random" decides both how easy it is to hide and how easy it is to catch.)

Chapter 2 - Tooling: Python, NumPy, and Image I/O (Week 2)

Try it

Goals: install the environment, run the end-to-end script, and read/write image arrays with NumPy. From now on every experiment can be verified immediately in the project. First we use a real photo to weld "image = array" into your head.

2.1 Installing the Environment

The project's dependencies are light: the core needs NumPy and Pillow, and the machine-learning part adds scikit-learn, joblib, etc. Use a fairly recent Python 3.9+ (readme example uses 3.14; older versions also work).

Install in the project root and do the first self-check

```
cd F:\Steganography
pip install numpy pillow
python src\test_core.py      # core algorithm self-test
python src\test_steg.py      # steganalysis self-test
python src\run_e2e.py        # end-to-end: embed -> decode -> analyze -> plot
```

Watch out

If the default `python` lacks tkinter, the GUI will not start. The README's fix is to use a tkinter-enabled interpreter (e.g. `C:\Python314\python.exe`); command-line scripts are unaffected. On `ModuleNotFoundError`, first confirm you installed into "the interpreter that runs it": `python -m pip install ...` is safer.

2.2 Thinking About Images with NumPy

NumPy arrays have four core concepts: **shape**, **dtype**, **slicing**, and **vectorized operations**. Steganographic algorithms essentially do: take an array -> rewrite the LSBs of some elements in some order -> store it back.

First look at "image = array" with a real photo:

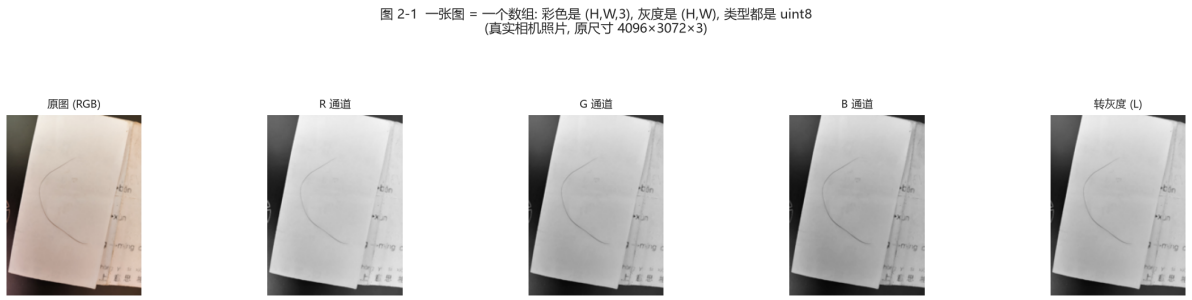

图 3: Fig. 2-1 image = array

Fig. 2-1 (real camera photo: color is a (H,W,3) three-channel array, grayscale is a (H,W) single channel, both dtype uint8. The R/G/B channels are each a "how bright is this color" grayscale image; grayscale is a human-luminance-weighted result)

Reading the figure:

- Color image: three 0-255 numbers stacked at each pixel, shape (H,W,3);
- Grayscale: one 0-255 per pixel, shape (H,W);
- Both are `uint8` (8-bit unsigned 0-255) - which is why "changing LSB" is bit ops rather than ±1.

The four concepts in 10 lines

```
import numpy as np
a = np.arange(12).reshape(3, 4)      # 3 rows 4 cols
print(a.shape, a.dtype)              # (3,4) int64
b = a[:, 1]                           # take column 2
c = a[0:2, 0:3]                       # top-left 2x3 block
x = np.zeros((64, 64), dtype=np.uint8)
x[10:20, 5:15] = 255                  # draw a white square
print((x & 1).sum())                  # count LSB=1
```

Note `dtype=np.uint8`: image pixels are 8-bit unsigned integers 0-255. With add/subtract beware **overflow wraparound**: `np.uint8(200) + 100` gives 44, not 300. The project flips LSB with "XOR 1" rather than "±1" precisely to avoid 0 underflowing to negative or 255 overflowing.

Fig. 2-2 (real computation: a uint8 array; uint8 wraparound - 200+100=44 not 300, so use 1 to flip bits; transpose/slice gives a "view" with non-contiguous memory; before saving to low-level libraries often need `np.ascontiguousarray`)

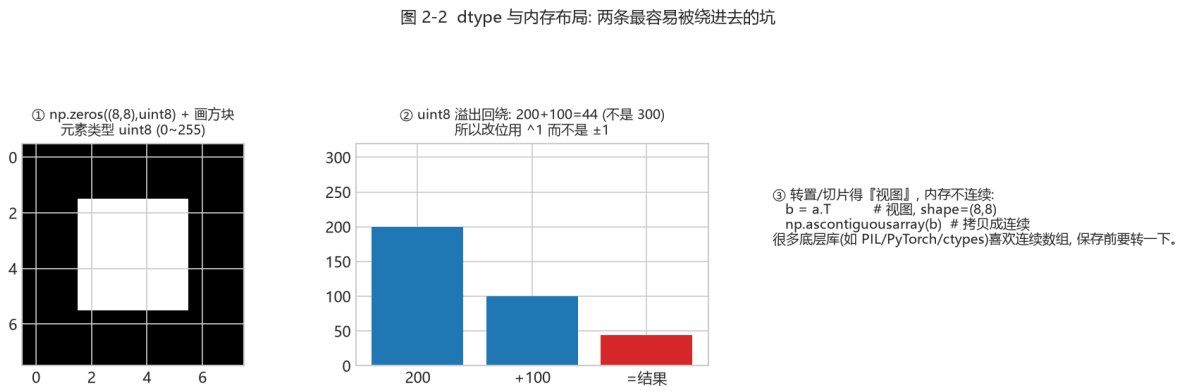

图 4: Fig. 2-2 dtype and memory layout

Try it

In an interactive shell type this and see for yourself that `np.uint8(200)+np.uint8(100)==44`, and the `flags` difference between `a.T` and `np.ascontiguousarray(a.T)`. Week 2's "can read code" starts with these small gotchas.

2.3 Image I/O: Meet image_io.py

The core interface of project/src/image_io.py

```
from PIL import Image
import numpy as np

def load_as_gray(path):
    im = Image.open(path).convert("L")   # force grayscale
    return np.asarray(im, dtype=np.uint8)

def save_image(image, path):
    im = Image.fromarray(np.ascontiguousarray(image))
    im.save(path)
```

Back to the code

Open `src/image_io.py` in full yourself; note the difference between `load_image()` and `load_as_gray()`, and the `SUPPORTED` extensions. Then run this to read the project's carrier image:

Hands-on: read an image, see shape, save a grayscale copy

```
import sys; sys.path.insert(0, "src")
import image_io as IO
```

```
img = IO.load_as_gray("img/cover.png")
print(img.shape, img.dtype)          # (256,256) uint8
print(img.min(), img.max(), img.mean())
IO.save_image(img, "output/my_copy.png")
```

Note `load_as_gray`'s `.convert("L")`: it forces any input to grayscale; while `save_image`'s `np.ascontiguousarr` guarantees contiguous memory. This "read as grayscale, write as contiguous" wrapper is the unified input interface for all downstream algorithms (embed/analyze/ML) - ensuring everyone works on the same-spec array.

2.4 First End-to-End Run: run_e2e.py

Run `python src/run_e2e.py`; the script does five things: generate a cover -> embed an English message with nsF5 p=3 (empty and keyed password) -> decode and compare -> blind-steganalysis both clean and stego images -> plot the code family/efficiency figures.

Try it

Read the terminal output and note three numbers: embedded bit count, changed-pixel count, and the clean vs stego analysis probabilities. Chapter 3 explains why they differ.

You do not need to understand embedding internals yet - Week 2's goal is only "get the code running on your machine" and build the "array LSB change" intuition. When you return to `train_model.py` after Chapter 8, you will see every "array operation" here serves a "feature vector" - that is the "learn by following the code" thread of this handbook.

2.5 Common Errors Cheat Sheet

Error / symptom	Cause	Fix
ModuleNotFoundError	installed into another interpreter	use the running interpreter's python -m pip install
image too small for header+body	fewer pixels than payload	use a larger image, or reduce p / shorten message
UnicodeDecodeError / garbled	ASCII-only by default	keep embedded text ASCII; see Chapter 10 for Chinese ideas
GUI exits / TclError	interpreter without tkinter	use a tkinter-enabled Python to run src/gui.py

Think about it

Why does `image_io` convert to ”contiguous array” (`np.ascontiguousarray`) when saving? Hint: slicing and transpose can produce non-contiguous memory views that many low-level libraries dislike. One layer deeper: since grayscale is a single (H,W) channel, why can Chapter 8’s ”features” reach 143 dimensions? (Hint: those are statistics, not pixels.)

Chapter 3 - LSB Steganography and Its Statistical Fingerprint (Weeks 3-4)

Try it

Goals: complete the smallest possible ”hide then detect” loop; understand what chi-square and RS analysis each measure; read the verdict logic in steganalysis.py. First we visually see how ”subtle” an LSB change is on a real carrier (img/cover.png), then see how statistics expose it.

3.1 Encryption vs Steganography: The First Question

Aspect	**Encryption**	**Steganography**
Protects	Message content: unreadable without a key	Communication: nobody sees there is a secret
Visibility	Ciphertext clearly ”not normal data”	Carrier looks completely normal
Typical use	Passwords, certificates, encrypted files	Covert channels, watermarking, forensics
Relation	Can encrypt then hide (recommended)	Steganography hides the ”existence” of a secret

Tip

A one-line rule: **encryption protects ”content”, steganography protects ”existence”.** Professional systems usually use both: encrypt the message first, then hide the ciphertext. Encrypted ciphertext looks near-random, which is less likely to reveal itself statistically.

3.2 Minimal Runable LSB Steganography

The simplest steganography is **LSB replacement**: write the secret bit string into the LSBs of pixels one by one. A grayscale image with M×N pixels can hide M×N bits.

Textbook minimal LSB (for understanding only; the project actually uses matrix coding from Chapters 4-5)

```python
import numpy as np

def text_to_bits(s):
    raw = s.encode("ascii")
    # 16-bit length header + 8 bits per character
    head = [(len(raw) >> i) & 1 for i in range(16)]
    body = np.unpackbits(np.frombuffer(raw, np.uint8))
    return np.concatenate([head, body]).astype(np.uint8)

def lsb_embed(cover, bits):
    flat = cover.ravel().copy()
    k = min(len(bits), flat.size)
    flat[:k] = (flat[:k] & 0xFE) | bits[:k]   # clear LSB then write
    return flat.reshape(cover.shape)

def lsb_extract(stego, nbytes):
    flat = stego.ravel() & 1
    return bytes(np.packbits(flat[16:16 + nbytes*8]))
```

Watch out

Lossy formats are LSB's enemy. PNG/BMP losslessly preserve bit planes; JPEG compression destroys LSBs. So the project saves PNG by default, and JPEG-domain hiding needs a separate route (the yccstego extension goes the quantized-DCT-coefficient route).

Why put a 16-bit length header first? Because the decoder must know "how many bytes to read" to recover the message. The project's `encode_string()` in `src/ns5_core.py` writes a 16-bit little-endian length first, then the ASCII body bits - you have replicated that structure in the 3.2 example.

3.2.1 First, See It: How "Subtle" Is an LSB Change?

On the project's demo carrier `img/cover.png`, do one real LSB replacement (hide random bits) - about 25% of pixels have their LSB rewritten. See what the image becomes:

Fig. 3-1 (real data: carrier `img/cover.png`; stego after hiding random bits via naive LSB; amplify |cover - stego| ×255 to finally see the changes scattered across the image; the carrier's own LSB plane - already near-noise)

Reading the figure:

- **vs look identical to the eye** - that is LSB's "concealment";
- **amplified ×255** reveals the noise scattered over the image - the "positions of changed pixels";

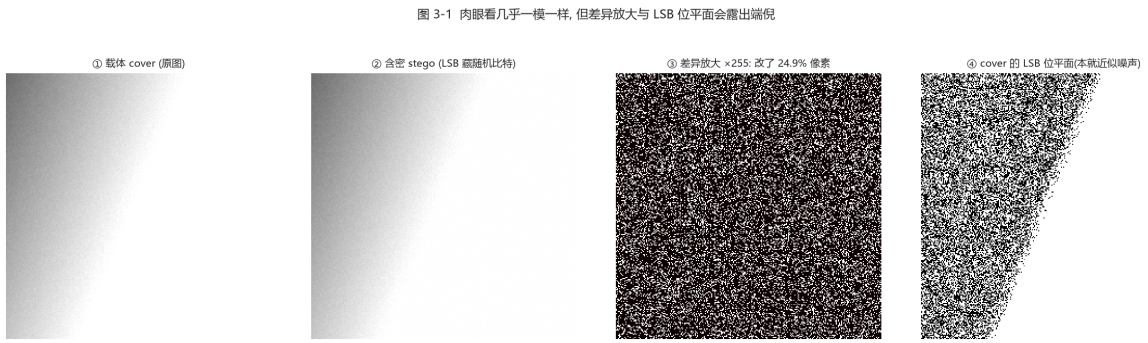

图 5: Fig. 3-1 cover vs stego vs amplified diff

- **cover's LSB plane** is already fine noise - so adding random bits there is hard to notice visually or by histogram.

3.2.2 Pixel Level: How Many Changed, and How?

Zoom into a small real block of pixel values and see each pixel's change before/after:

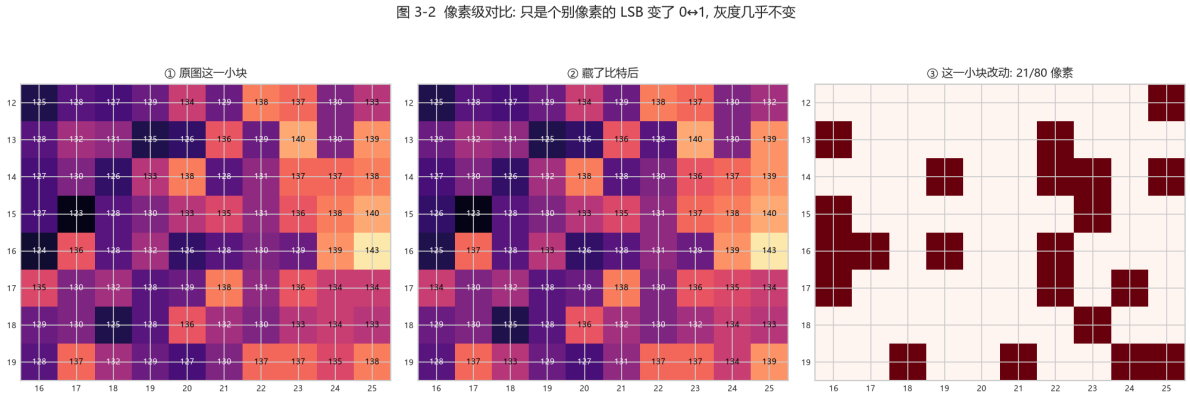

图 6: Fig. 3-2 pixel-level comparison

Fig. 3-2 (real data: / the same small region's gray values before/after embedding; marks the 21/80 pixels changed. Note every change differs by only 1 - because only the LSB flips 0 1)

Reading the figure:

- Each changed pixel's value moves by only ±1 (e.g. 127→128, 135→134), because only the **lowest bit** changes;
- The eye is completely insensitive to a 1-level gray change - that is "visual redundancy";
- But exactly this "pair-flattening" change leaves a measurable fingerprint for statistical detection (3.3 chi-square, 3.4 RS).

Tip

Remember two words: **visual redundancy** (the eye cannot see) and **statistical redundancy** (a detector can see). LSB hiding uses the former but breaks the latter - that becomes the breakthrough for all statistical detection.

3.3 Why It Is Exposed: The Chi-Square Test

In natural images the adjacent gray values 2i and 2i+1 (e.g. 100 and 101) usually appear with **unequal** counts. LSB replacement forces the even/odd pair to "flatten": when a pixel changes from 2i to 2i+1 or back, the two gray counts tend to equalize. Westfeld's chi-square test does exactly this:

1. Count the 256-level grayscale histogram and pair adjacent values (0,1),(2,3),...,(254,255);
2. Assuming "embedded", the two counts in each pair should be nearly equal; the expected value is half the pair sum;
3. Compute $\Sigma(\text{observed-expected})^2/\text{expected}$ (only over pairs with positive count);
4. Convert to a p-value: high p means "consistent with the uniform assumption" $\rightarrow$ suspect LSB randomization;
5. The project also cuts the image into 20 prefix segments, computes a p per segment, and takes the median for a steadier verdict.

Note the p-value meaning is easy to get backwards: here a larger p is more suspicious, because it says "if the LSB were fully randomized, seeing this distribution is likely" - a clean smooth image usually is not so uniform.

This compares the adjacent-gray-pair (2i, 2i+1) counts for a clean image and after LSB embedding: on the left each pair's even/odd counts are clearly uneven; after embedding the two columns are "flattened". That is exactly the signal chi-square catches - the more balanced the counts, the higher the p, the more suspicious.

Watch out

A naturally noisy image can fool chi-square. Digital photos already have near-random LSBs, so the chi-square p is naturally high. So the project adds a "content randomness" correction: first estimate how random the image inherently is via the grayscale difference entropy, then decide whether to treat a high p as evidence of embedding. This is a key engineering detail that makes the heuristic usable.

3.4 RS Analysis: The "Structural Collapse" of the LSB

Fridrich et al.'s RS analysis takes a different angle: instead of comparing gray-pair counts, it looks at the **spatial structure** of the LSB plane. It cuts the pixel stream into groups of 4, defines a

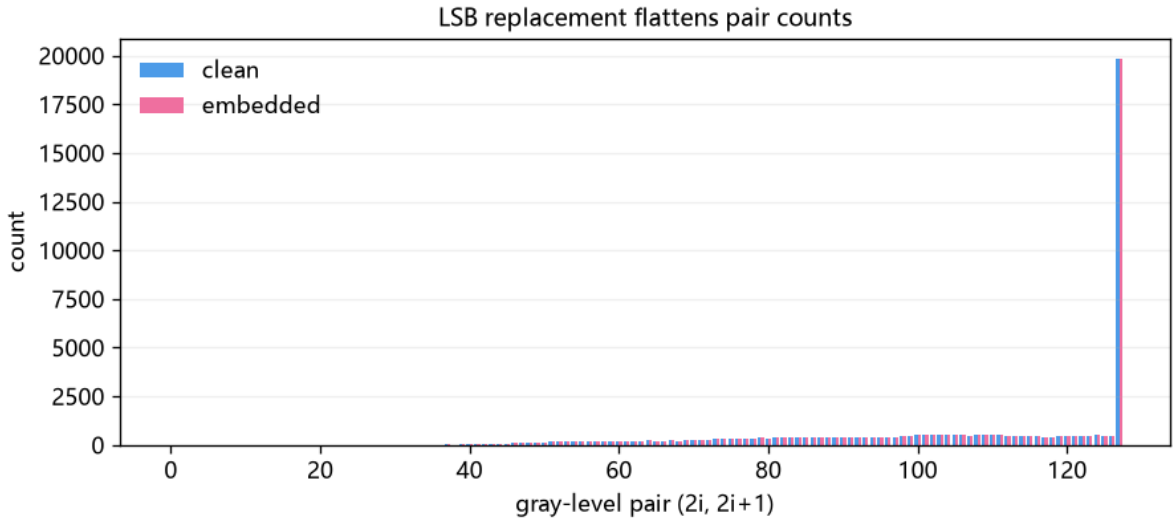

图 7: fig-10

smoothness discriminant f (sum of absolute adjacent differences within the group), then uses a ”positive mask” and ”negative mask” to flip some LSBs in the group, counting groups where f increases (Regular R) or decreases (Singular S) after flipping.

A clean natural image has a clear ”regular tendency” under the negative mask, quantified by Gn = (Rn-Sn)/N which is clearly positive (the project's clean smooth images often 0.3-0.7). LSB replacement randomizes the plane, collapsing this structure - Gn drops markedly. The project also reports Gr, estimated embedding rate, etc., together forming the ”RS evidence”.

The core RS logic in the project's steganalysis.py (pseudocode, only illustrating the grouping/statistical idea)

```
groups = flat[:m].reshape(-1, 4)          # group of 4 pixels
fg = np.abs(np.diff(groups, axis=1)).sum(axis=1)   # original smoothness f
pos = (groups ^ np.array([0,1,1,0])) & 0xFF        # +M flip
neg = (groups ^ np.array([1,0,0,1])) & 0xFF        # -M flip
Rm = (f(pos) > fg).sum();  Sm = (f(pos) < fg).sum()
Rn = (f(neg) > fg).sum();  Sn = (f(neg) < fg).sum()
Gn = (Rn - Sn) / n          # clearly positive for clean, ~0 after randomization
```

This uses the project's real statistics to compare clean (blue) vs stego (pink): the RS gap Gn collapses from clearly positive (structure), while the chi-square p and ML probability rise. LSB embedding destroys the LSB plane's ”order” - this bar chart is a direct comparison of the three lines of evidence (RS / chi-square / ML).

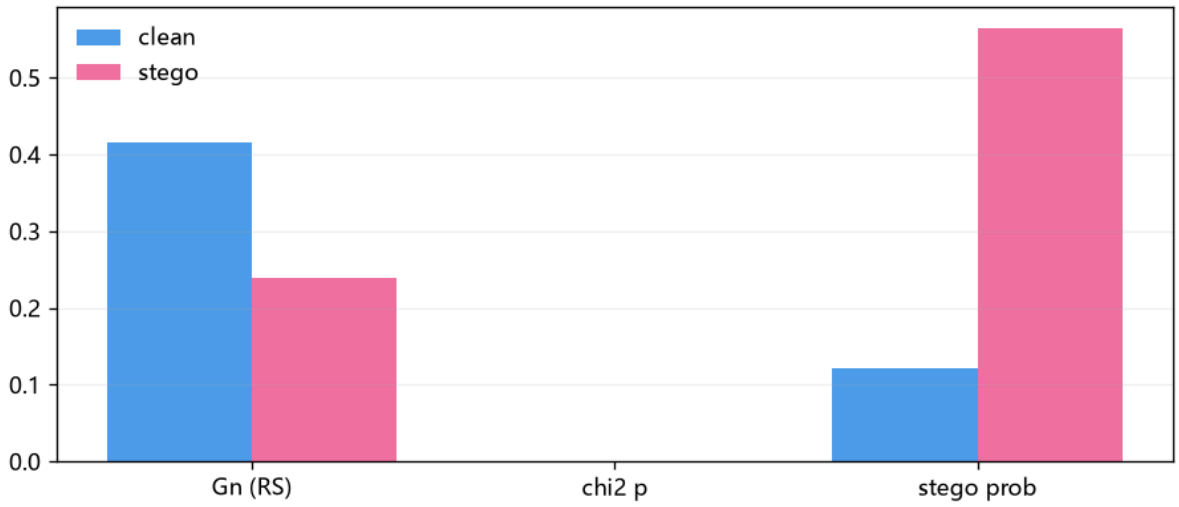

图 8: fig-11

3.5 The Project's Combined Verdict: analyze()

`src/steganalysis.py`'s `analyze(image, sensitivity=...)` does not look at one statistic, but combines four signals:

- median chi-square p (are gray pairs flattened?);
- RS collapse amplitude (how far Gn drops relative to a content-adaptive baseline);
- grayscale difference entropy and LSB difference entropy (is the randomness native to the image?);
- the prefix p sequence (is the randomization local or global?).

Three sensitivity levels (strict / balanced / loose) tune the "possible / highly possible" thresholds and the probability-pull coefficient, essentially choosing an operating point between **false positives** and **misses**.

Look ahead to Chapter 7

Chapter 7 upgrades this heuristic into "features + a supervised classifier" and draws fuller ROC / AUC (Fig. 7-6) and feature distributions (Fig. 8-1/8-2).

3.6 Hands-On: Hide a Sentence, Then Catch It

Using the real project API: embed -> save -> analyze (run cell by cell in Jupyter)

```
import sys; sys.path.insert(0, "src")
import numpy as np
```

```
import image_io as IO
from ns5_core import embed_string, extract_string
import steganalysis as SA

clean = IO.load_as_gray("img/cover.png")
stego, report, nbits = embed_string(
    clean, "Hello␣nsF5!", method="nsF5", p=3, password="")
IO.save_image(stego, "output/lab3_stego.png")
print("changed␣pixels:", report["cover_changed"])
print("decoded:", extract_string(stego, method="nsF5", p=3))
for name, im in (("clean", clean), ("stego", stego)):
    r = SA.analyze(im)
    print(name, "Gn=%.3f" % r["RS_Gn"],
          "chi2p=%.3f" % r["chi2_pvalue"],
          "prob=%.2f" % r["stego_probability"], r["verdict"])
```

Try it

Change the message to 5000 characters and run again (see `src/test_steg.py`); watch the changed-pixel ratio, Gn drop, and stego probability change. Then swap in a noisier photo and see whether chi-square and RS can still distinguish.

3.7 Summary and Self-Check

- LSB embedding uses **visual redundancy** (Fig. 3-2: each change differs by 1), invisible to the eye (Fig. 3-1);
- LSB replacement flattens adjacent gray-pair counts (chi-square) and destroys LSB-plane structure (RS);
- A high p is not ”safe”; first judge whether the image is naturally random;
- False positives vs misses are a trade-off; the sensitivity setting picks an operating point;
- The project’s analyze() is an engineering combination of ”statistical signals + content baseline”.

Think about it

Plain LSB replacement hides 1 bit/pixel with an average 50% change rate. If you hide only a small message and change less than 1% of pixels, can chi-square still catch it? RS? Take this question into Chapter 4. One layer deeper: Fig. 3-1 shows the LSB plane is already near-noise - so why does chi-square still catch it? (Hint: the key is ”a clean image’s even/odd counts are initially uneven”, not ”the LSB looks random”.)

Chapter 4 - Matrix Embedding and F5: Change Less, Hide More (Week 5)

Try it

Goals: understand why "embedding efficiency" matters; hand-compute a p=3 Hamming syndrome embedding example; read the matrix-coding schematic and efficiency curves; know F5's "shrinkage" problem.

4.1 From "1 bit per pixel" to "p bits per change on average"

Naive LSB changes 0.5 pixels on average per stored bit - very inefficient. Suppose there were a code where each block of n pixels lets you store p bits by **changing at most 1 pixel**: changes drop dramatically. That idea is **Matrix Embedding**, and its math tool is the binary Hamming code.

Define embedding efficiency α = average bits carried per modification. The theoretical ceiling is p bits per change, $\alpha(p) = p \cdot 2^p/(2^p - 1)$: 2.67 at p=2, 3.43 at p=3, 4.27 at p=4, rising with p - but the block length $n = 2^p - 1$ grows too.

Tip

Why is larger p "better"? Because the Hamming code uses $n = 2^p - 1$ positions to carry p message bits; the bigger p, the less "waste" of n relative to p, and the more info each change carries on average. The cost: bigger blocks, and the message is pickier about the carrier structure. Look at Fig. 4-1 to see "7 positions hold 3 bits, flipping just 1 pixel" directly.

4.2 Hamming in a Nutshell: Parity-Check Matrix and Syndrome

Let block length $n = 2^p - 1$. Build a $p \times n$ **parity-check matrix H** whose n columns are exactly all non-zero vectors in $GF(2)^p$: at p=3 the seven columns are binary 1..7. Write the n pixel LSBs of a block as a column vector x, and define the syndrome $s = H \cdot x \pmod 2$, a p-bit vector.

To embed p message bits m, we want to flip a few bits so the new syndrome equals m. Since every column of H is distinct, flipping bit j only turns the syndrome into $s \oplus H_j$ - that is, **any needed syndrome change matches some column**.

1. Compute the current syndrome $s = H \cdot x$;

2. If s == m, no change needed in the block;
3. Otherwise compute the difference d = s ⊕ m (a non-zero p-bit vector);
4. Find the column of H equal to d - call it column j;
5. Flip the LSB of pixel j; done (H·x = m).

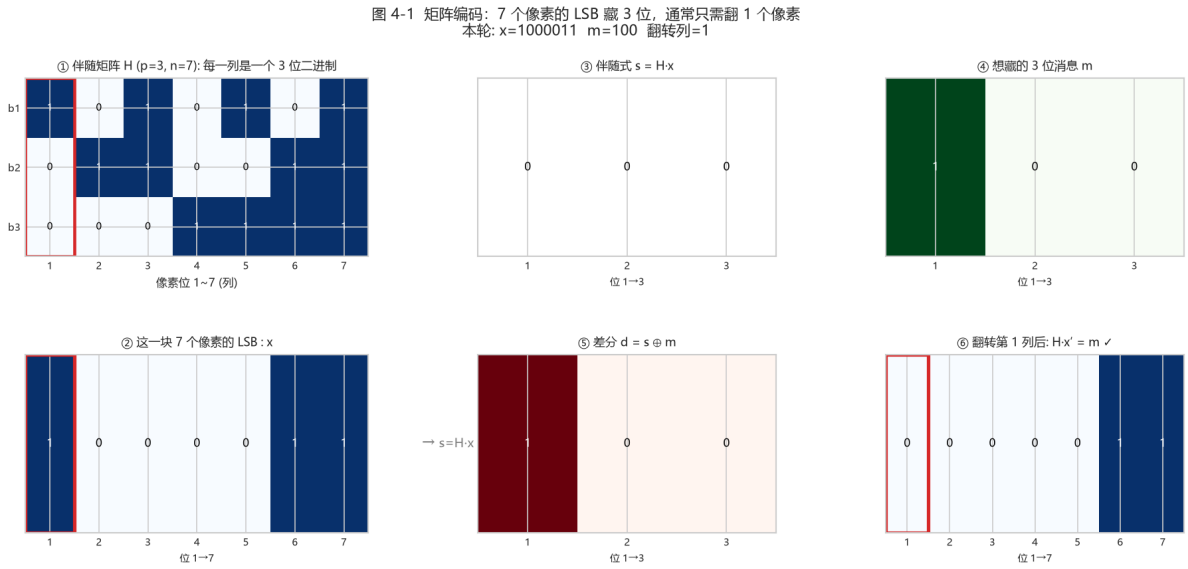

图 9: Fig. 4-1 matrix-coding syndrome schematic

Fig. 4-1 (real computation: parity-check matrix H (p=3, n=7); the red box highlights the "target column"; this block's 7 pixel LSBs: x; current syndrome s=H·x; the 3 message bits to hide: m; difference d=s⊕m. Since d equals H's column 1, only the 1st pixel's LSB is flipped to get the new syndrome = m)

Read this figure and you have all of matrix coding:

- **H has 7 columns, each a 3-bit binary** (for 17);
- **s = H·x**: XOR the columns where x is 1 to get the current syndrome;
- **To turn the syndrome into m**: just flip the pixel whose column equals d=s⊕m;
- Conclusion: **to hide 3 bits, most cases only need 1 pixel changed**.

4.3 A Worked p=3 Example

Let the 7 pixel LSBs of a block be x = [0,1,0,1,1,0,1]. Columns in binary order (low bit on top): H =[1,0,0], H =[0,1,0], H =[1,1,0], H =[0,0,1], and so on. First the syndrome: the 1-positions of x are columns 2,4,5,7; XOR them: H ⊕H ⊕H ⊕H = [0,1,0]⊕[0,0,1]⊕[1,0,1]⊕[1,1,1] = [0,0,1], so s=[0,0,1].

To embed m=[1,1,0]: d = s⊕m = [0,0,1]⊕[1,1,0] = [1,1,1]. H =[1,1,1], hits column 7, so flip the 7th pixel's LSB: x =[0,1,0,1,1,0,0]. Recompute s =H·x =[1,1,0]=m, success.

7 pixels carry 3 message bits, but on average only 87.5% of blocks need a 1-bit change - about 0.875 changes per block, i.e. per 3 bits about 0.875 changes, 3.43 bits per change.

Try it

`src/matrix_demo.py` turns this into a clickable visual panel (the "Matrix Coding Demo" tab in the GUI). Use `demo_step(p, x, m_bin)` to see the returned s, d, and the matched column, then verify H·x == m by hand.

4.4 Why Efficiency Matters: Capacity, Change Rate, and Detectability

"Hiding successfully" is not the skill; what matters is **for the same message, the fewer changes, the smaller the statistical trace, the harder to detect**. Let us plot the three quantities vs p with the real formulas - then you see the matrix-coding trade-off.

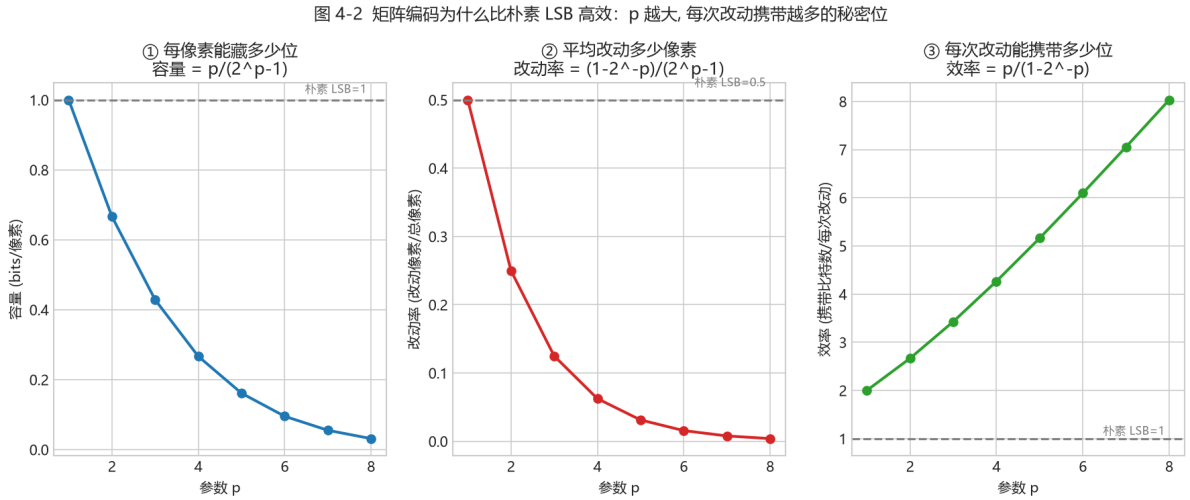

图 10: Fig. 4-2 embedding efficiency

Fig. 4-2 (real computation: as p grows - capacity per pixel p/(2 -1) drops; average change rate (1-2)/(2 -1) drops sharply; bits carried per change (embedding efficiency) p/(1-2) rises. Dashed lines are the naive-LSB baseline: capacity 1, change rate 0.5, efficiency 1)

Reading the figure:

- **Change rate** ( ) directly measures "trace": the larger p, the fewer pixels changed to hide the same message, the harder statistical detection catches it;

- **Efficiency** ( ) measures ”value per change”: p=3 is 3.43 bits/change, over 3x naive LSB;
- **Cost** ( ): the larger p, the smaller per-pixel capacity; hiding the same message needs a larger carrier.

Put them on one ”capacity vs change-rate” trade-off plot and the trade-off is obvious:

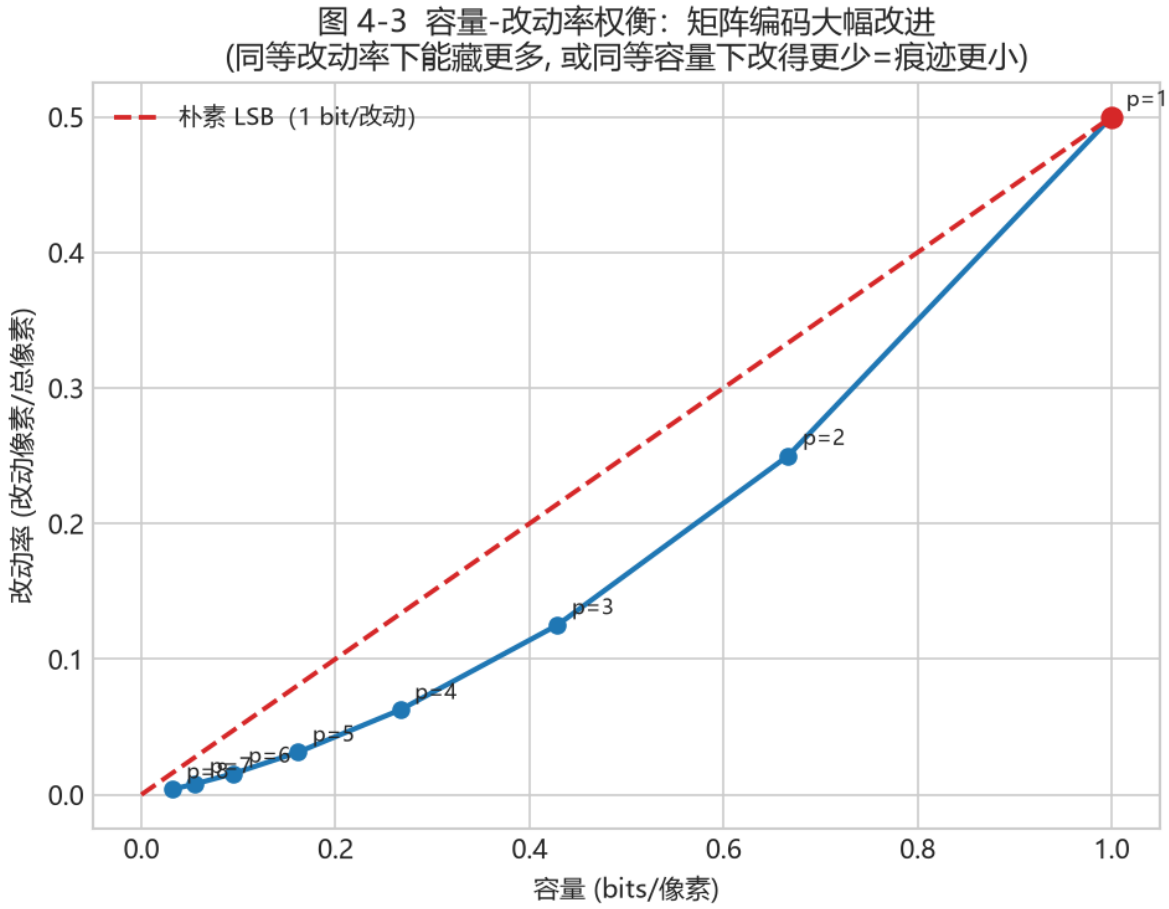

图 11: Fig. 4-3 capacity-change-rate tradeoff

Fig. 4-3 (real computation: blue line is matrix coding at different p (capacity on x-axis, change rate on y-axis); red dashed line is naive LSB. The whole matrix-coding curve sits much lower - ***at the same change rate it hides more, or at the same capacity it changes less and leaves a smaller trace****. That is why nsF5 in Chapter 5 insists on matrix embedding)*

Tip

Steganography performance is usually measured by two things: **capacity** (how much) and **distortion/change rate** (how detectable). Matrix embedding’s meaning is to **raise capacity at the same distortion, or lower distortion at the same capacity**. This ”change less = safer” idea runs through the nsF5 of Chapter 5 and the detection of Chapter 8.

4.5 F5: Matrix Embedding + JPEG Coefficient Decrement

F5 (Westfeld) is a JPEG steganographic scheme: it does matrix embedding on quantized DCT AC coefficients. Changing a "bit" is not an LSB flip but reducing the coefficient's **absolute value by 1** (e.g. +3→+2, -5→-4), which both flips the LSB parity and avoids obvious statistical anomalies.

The problem is "shrinkage": when a coefficient with absolute value 1 is decremented, it becomes 0. A 0 coefficient in JPEG decoding means "no such frequency component", so F5 treats it as no longer a legal carrier, drops the block, and retries later message bits. **Shrinkage** reduces the actual payload, lowers the achieved efficiency below the theory, and leaves a detectable statistical feature.

4.6 Back to the Project: MatrixEmbedding

Back to the code

Open the `MatrixEmbedding` class in `src/ns5_core.py`: `_embed()` is literally the 4.2/4.3 steps - compute s, compare m, find the column, flip. Fig. 4-1/4-2/4-3 are the visualizations of the math behind that code.

MatrixEmbedding._embed() core (real code excerpt)

```
xl = (c[pos] & 1).astype(np.uint8)      # take block LSBs
s  = syndrome(H, xl)                      # s = H·x
if np.array_equal(s, m): continue         # exactly matched, no change
d = (s ^ m).astype(np.uint8)              # difference
col = int(np.where(np.all(H == d[:, None], axis=0))[0][0])
c[pos[col]] ^= 1                          # flip the matched pixel
```

Back to the code

`src/efficiency.py`'s `plot_code_family_and_efficiency()` also draws the code family and efficiency plot (saved to `output/efficiency.png`), cross-checkable with Fig. 4-2.

4.7 Summary and Self-Check

- Matrix embedding works in blocks: n positions carry p bits, at most 1 change (Fig. 4-1);
- Distinct H columns ⇒ any syndrome difference locates the unique pixel to flip;
- Embedding efficiency α=p·2 /(2 -1), rising with p but block length grows too (Fig. 4-2);
- **Capacity-change-rate trade-off**: the fewer changes, the harder to detect (Fig. 4-3) - the fundamental answer to "why matrix coding";
- F5 embeds by decrementing JPEG coefficients; shrinkage occurs when |coefficient|=1 becomes 0.

Think about it

If F5 skips and retries on shrinkage, can the message still decode correctly? How does the decoder know which blocks were skipped? Do not worry if stuck - Chapter 5's nsF5 uses wet-paper coding to sidestep this entirely. One layer deeper: using Fig. 4-3, if I want to hide only 0.2 bit/pixel, how low can matrix coding push the change rate (versus naive LSB's 0.1)? That directly sets how hard steganalytic detection is.

Chapter 5 - nsF5 and Wet Paper Coding: The Project's Core Algorithm (Week 6)

Try it

Goals: explain why shrinkage damages efficiency; understand wet paper coding as "wet positions untouched, solve on dry positions"; walk through the complete nsF5 flow in ns5_core.py. Two real-computed figures (Fig. 5-1 / Fig. 5-2) are included.

5.1 First, the Problem: F5's Shrinkage

F5 modifies JPEG quantized coefficients by decrement: each time a coefficient's absolute value drops by 1, its LSB parity flips. But a coefficient with absolute value 1 (e.g. +1, -1) becoming 0 upon decrement. To JPEG, a 0 coefficient is an "absent frequency band", unsuitable as a carrier, so F5 abandons the block and looks elsewhere - that is **shrinkage**.

- Shrinkage makes the **actual capacity** lower than the theory: blocks are wasted, the message needs more positions;
- Shrinkage makes the **embedding efficiency** lower than $\alpha(p)$: more modifications are needed to compensate;
- Shrinkage also leaves a detectable statistical feature: more coefficients are changed to 0, and the histogram looks abnormal.

The project makes a clever analogy in the spatial (pixel) domain: subtract 128 from a pixel to get a "signed value" xv = pixel - 128, so pixels with $|xv| \leq 1$ (127, 128, 129) are the "dangerous" positions where decrement would hit near zero. The shrinkage problem of matrix embedding in JPEG becomes, in the pixel domain, the same problem as "127/128/129 must not be ordinary modification targets".

5.2 Wet Paper Coding: Mark the Wet First, Then Solve

Wet Paper Coding is a framework by Fridrich et al. that answers: if some positions of a carrier are "untouchable" (like paper soaked wet - writing on them is invisible), can we complete embedding

using only the remaining ”dry” positions?

Yes - and without the two parties needing to agree in advance which positions are wet, as long as the algorithm can reconstruct the same wet/dry partition from the image content itself. That is exactly what the project does:

1. For each block, find the positions that would become zero/dangerous on decrement and mark them wet;
2. The real problem: find a modification vector e on the **dry** positions such that flipping them makes the block's syndrome equal the target m;
3. That is, solve H[:,dry] · y = d, where d = s⊕m is the desired syndrome change;
4. The project solves in the order ”single dry column hit → two dry columns XOR hit → GF(2) Gaussian elimination fallback”, changing as little as possible;
5. The solution y is a 0/1 vector: y=1 positions are the dry positions actually to decrement.

图 5-1 nsF5 的「湿点/干点」：先把碰不得的位置划出来

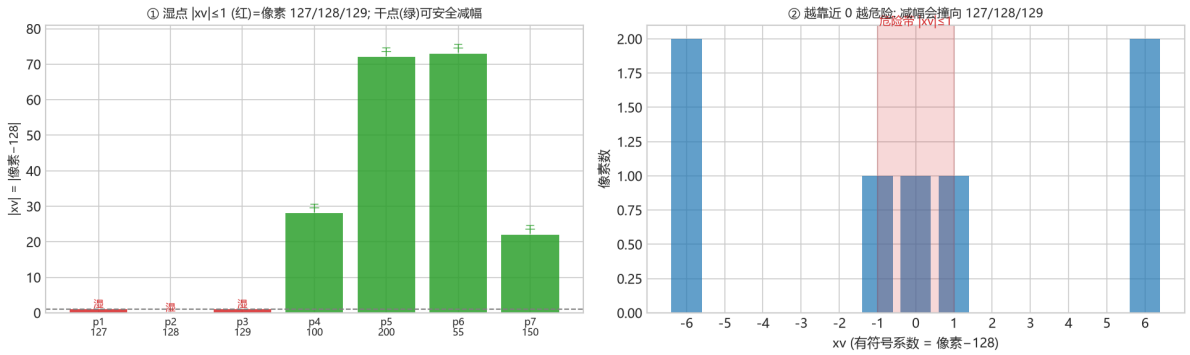

图 12: Fig. 5-1 wet/dry partition

*Fig. 5-1 (real computation: a p=3 block; pixels 127/128/129 are judged **wet**(red) because ”|xv|≤1, decrement would hit toward 0”, the rest are **dry**(green); mapping pixel-128 to the signed coefficient xv - the closer to 0, the more dangerous)*

Reading the figure

The first step is always ”**mark the wet first**”: exclude positions that would become illegal (0 coefficient / hit zero) after decrement. Then embedding only happens at safe positions - that is exactly why nsF5 does not shrink and does not retry.

Tip

Why does "marking wet first" remove shrinkage? Because shrinkage comes from "only discovering it became 0 halfway through". nsF5 excludes all zero-colliding positions up front and solves only on safe dry points, so every block can embed successfully - no retries, no extra 0 coefficients.

5.3 Reading the Project: nsF5Pixel._embed()

In `src/ns5_core.py`, the `nsF5Pixel` class inherits from `MatrixEmbedding` and only overrides `_embed()`. Reading it line by line shows the full nsF5 decision tree:

nsF5Pixel._embed() decision tree (pseudocode, logic identical to the source)

```
xv = c[pos].astype(np.int16) - 128      # pixel -> signed coefficient
s = syndrome(H, (xv & 1).astype(np.uint8))
if s == m: continue                     # syndrome already matched
tc = hit_column(H, s ^ m)               # most "direct" candidate position
if abs(xv[tc]) > 1:                     # decrement won't hit zero -> change
   directly
    c[pos[tc]] -= sign(xv[tc])          # step toward 128
else:                                   # candidate position is wet
    dry = [k for k in range(n) if abs(xv[k]) > 1]
    e = solve_wet_paper(H, dry, d)      # solve only on dry points
    if e: decrement all hit dry points per e
    else: fallback flip target LSB (ensure decodability)
```

5.4 The Three Solver Functions

Function	Role	Key point
solve_wet_paper(H, dry_cols, target)	find a sparse solution among dry columns	weight 1 → weight 2 → Gaussian fallback
gauss_solve_GF2(cols, b)	GF(2) Gaussian elimination	free variables set to 0 to reduce changes
permute_index(total, seed)	deterministic pseudo-random permutation	splitmix64 + Fisher–Yates, C++-accelerable

5.4.1 A Computable GF(2) Example (p=3, enough dry columns)

Let p=3. Column j of `build_hamming(3)` is the 3-bit expansion of the binary number j (low bit on top), written $h_1, \ldots, h_7$:

$$H = \begin{bmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{bmatrix}, \qquad h_1 = \begin{smallmatrix}1\\0\\0\end{smallmatrix}, h_2 = \begin{smallmatrix}0\\1\\0\end{smallmatrix}, h_3 = \begin{smallmatrix}1\\1\\0\end{smallmatrix}, h_4 = \begin{smallmatrix}0\\0\\1\end{smallmatrix}, h_5 = \begin{smallmatrix}1\\0\\1\end{smallmatrix}, h_6 = \begin{smallmatrix}0\\1\\1\end{smallmatrix}, h_7 = \begin{smallmatrix}1\\1\\1\end{smallmatrix}$$

Assume pixels 1, 2, 3 of the block are wet (`|xv|≤1`, i.e. 127/128/129), so the dry columns are $\{4, 5, 6, 7\}$. Suppose the desired syndrome difference is $d = s \oplus m = (1, 1, 0)^\top$.

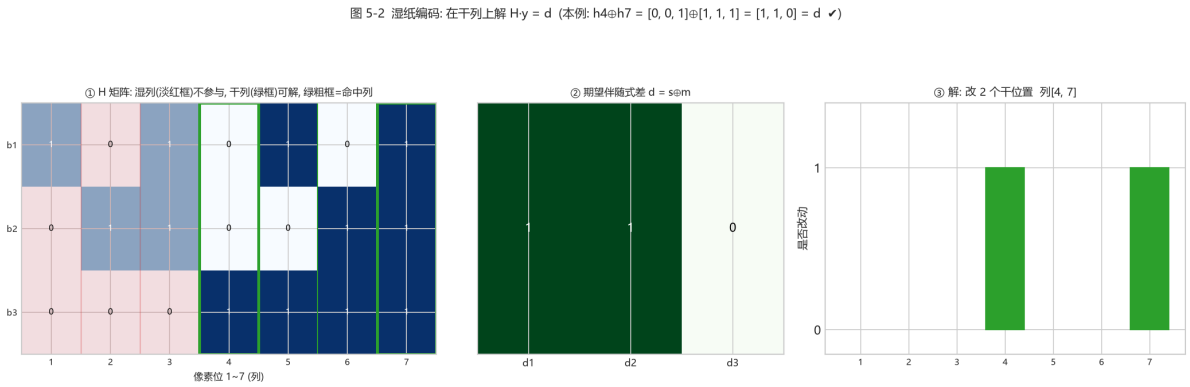

图 13: Fig. 5-2 wet paper solving

Fig. 5-2 (real computation: H matrix - wet columns (pink boxes) excluded, dry columns (green boxes) usable, thick green = matched columns; the desired syndrome difference d; the solution - find as few dry columns as possible whose XOR equals d. Here no single column hits; the 2-column hit h ⊕h = d, so columns 4 and 7 are changed)

Reading the figure

- **Single-column hit**: see if any dry column equals d (here none); - **2-column hit**: see if two dry columns XOR to d (here h ⊕h =[0,0,1]⊕[1,1,1]=[1,1,0]=d, hit); - If neither works, use **GF(2) Gaussian elimination** as fallback. Key: solve only on "dry" points; wet points are never touched.

Step 1, single-column hit: check whether any dry column equals $d = (1, 1, 0)^\top$: $h_4 = (0, 0, 1)^\top$, $h_5 = (1, 0, 1)^\top$, $h_6 = (0, 1, 1)^\top$, $h_7 = (1, 1, 1)^\top$. None equals $(1, 1, 0)^\top$ → no single hit.

Step 2, two-column XOR: find two dry columns XORing to d. For example $h_5 \oplus h_6 = (1, 0, 1)^\top \oplus (0, 1, 1)^\top = (1, 1, 0)^\top = d$. **Hit!** So only these two dry positions (their coefficients) are changed, no Gaussian needed. (The code finds the first matching pair in random order and may actually take h ⊕h , which is equivalent to h ⊕h ; both satisfy `syndrome(H,e)=d`.)

5.4.2 When Is Gaussian Needed? - When Dry Columns Span All of GF(2)

The dry columns $\{4, 5, 6, 7\}$ above are 4 in number and span all of $\mathrm{GF}(2)^3$, so most d can be handled by a single or double hit. But if there are too few dry columns, or they are linearly dependent, the span shrinks.

Example: only dry columns $\{4, 5\}$ (more wet points). They and their pairwise XORs cover only 4 syndromes: $(0, 0, 0)$, $h_5 = (1, 0, 1)^\top$, $h_6 = (0, 1, 1)^\top$, $h_5 \oplus h_6 = (1, 1, 0)^\top$. If $d = (0, 0, 1)^\top$, none of these covers it - **no solution**.

Why? GF(2) Gaussian elimination packs the dry columns into $A = [h_5\ h_6]$, a 3×2 matrix of rank at most $2 <$ p=3, so it **cannot span 3 dimensions**. A solution exists only if $d \in \text{span}(A)$; $d = (0, 0, 1)^\top \notin \text{span}\{(1, 0, 1)^\top, (0, 1, 1)^\top\}$, so `gauss_solve_GF2` returns `None` and `solve_wet_paper` returns `None`. **This is the mathematical essence of "not enough dry points": rank $<$ p $\Rightarrow$ some message bits cannot be realized.**

Solvability criterion

To embed any p-bit message on dry points, the number of dry columns usually needs to be $\geq$ p and those columns must span all of GF(2) . That is why wet paper coding needs enough dry points (in nsF5 dry points are almost always sufficient; in extreme cases the code falls back to flipping the target bit to guarantee decodability).

5.4.3 Setting Free Variables to 0 = Change as Little as Possible

Solving $A\,y = d$ generally has infinitely many solutions (when columns exceed the rank). `gauss_solve_GF2` sets **free variables to 0**, equivalent to "changing only the necessary dry columns", thereby **reducing the number of changed pixels** - which both raises embedding efficiency and lightens the statistical trace.

Back to the code

Open lines 145-215 of `src/ns5_core.py` and read against the 5.4 table. Read the function comments first, then verify with `test_wet_paper_needed()` in `src/test_core.py`: it forces pixels to 127/128/129 to create wet points and still completes an embed/decode roundtrip.

5.5 Verification: Efficiency and Roundtrip

Run the core self-tests + compare the two methods

```
python src\test_core.py
# compare modified-pixel counts for matrix vs nsF5:
import sys; sys.path.insert(0, "src")
import numpy as np; from ns5_core import embed_string
img = np.random.default_rng(0).integers(0, 256, (64, 64), np.uint8)
for method in ("matrix", "nsF5"):
    stego, rep, nb = embed_string(img, "Hello␣nsF5!", method=method, p=3)
    print(method, "changed", rep["cover_changed"], "pixels␣/␣capacity", nb)
```

On the same random image and same message, matrix and nsF5 do not necessarily differ in change count - because nsF5's protected range (127/128/129) rarely triggers on random images. The real difference appears on an image dense with "dangerous pixels": nsF5 still embeds without shrinkage, while matrix may repeatedly fail.

Try it

Replace `img` with an image that is entirely 127/128/129, then compare the two methods' success count and change count - you directly feel the value of wet paper coding.

5.6 Summary and Self-Check

- Shrinkage = decrement hitting zero and discarding a block, harming capacity, efficiency, and statistical safety (Fig. 5-1's "danger zone");
- Wet paper coding marks untouchable positions as wet up front and solves the syndrome equation only on dry points (Fig. 5-2);
- nsF5 = matrix embedding + wet paper coding, no shrinkage, no retry;
- The project simulates JPEG coefficients in the pixel domain with xv = pixel-128, treating $|xv| \leq 1$ as wet.

Think about it

Why does wet paper coding not need a pre-agreed set of wet positions? How does the decoder know which pixels cannot carry message bits? Hint: the decoder only reads LSBs to compute the syndrome - it never needs to know who was bypassed during embedding. One layer deeper: if a block has too many wet points and the dry ones do not span all of GF(2) , what happens? (Combine with the rank argument in 5.4.2.)

Chapter 6 - Passwords, SHA-256, and Keyed Security Design (Weeks 6-7)

Try it

Goals: understand why embedding positions must depend on a key; explain decoder self-synchronization and tamper perception; know the limits of this design. Two real-computed figures (Fig. 6-1 / Fig. 6-2) are included.

6.1 What If the Embedding Positions Are Fixed?

Suppose you always embed sequentially from the top-left. Knowing the algorithm, an attacker can: read the LSBs of the first N pixels and recover the message; copy the message in place onto another "lookalike" image (position-replacement attack); perturb a fixed region to destroy your message while leaving the rest untouched.

So modern steganography makes the embedding path a **key-controlled pseudo-random order**: without the password (or the content hash), nobody knows where the message is, and positional attacks are infeasible.

Fig. 6-1 (real computation: using the project's `permute_index()` splitmix64 + Fisher-Yates, drawn as "visit order" on a 64x64 grid. Left is password A, right is password B - the two "orders" are entirely different, both looking like random noise)

Reading the figure

Brighter color = visited earlier. Change the password and the entire order changes - an attacker without the password cannot predict "where the message hides". That is the meaning of **keying**: **security comes from "not knowing the positions", not from "being invisible".**

6.2 Two Seed Rules: Recognize the Image First, Then Find the Payload

The project splits pixels into two pools: the **header pool** (the first N_h blocks) and the **body pool** (the rest). Key derivation uses two independent seeds:

图 6-1 嵌入位置是「由密钥决定的伪随机顺序」：两张不同口令给出完全不同的路径

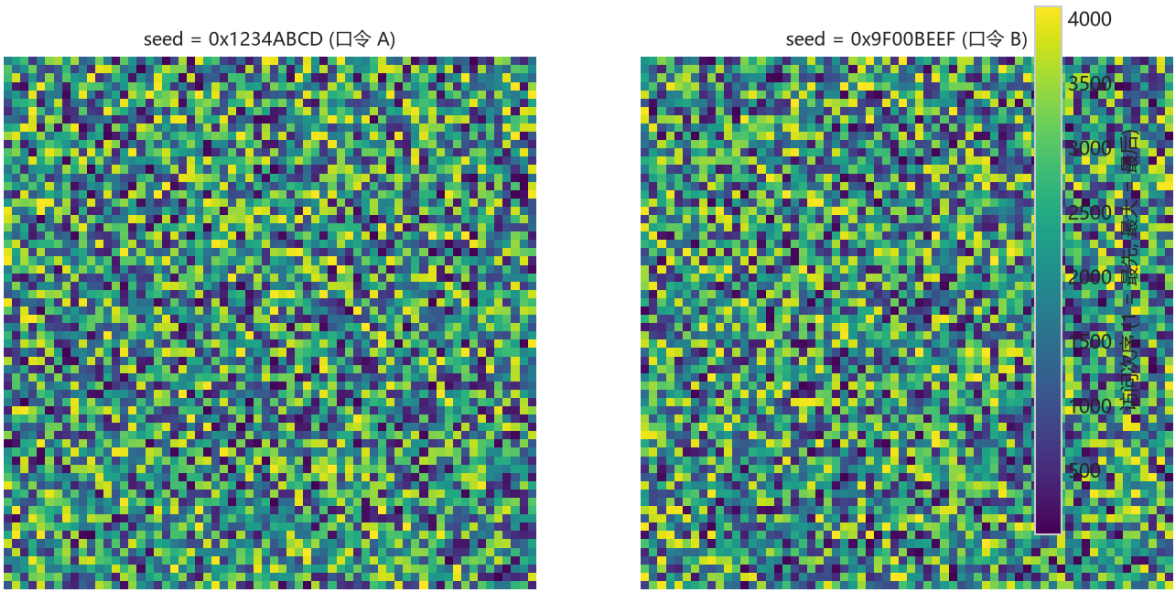

图 14: Fig. 6-1 keyed permutation path

- **seed0 = derived from the password only**: determines the header-pool shuffle order. The header embeds cover_hash - the first 16 bytes (128 bits) of a SHA-256 of the original image bytes;
- **seedB = cover_hash + password combined**: determines the body-pool shuffle order and block partition.

To decode, the receiver first rebuilds seed0 with their own password, reads back cover_hash from the header, then rebuilds seedB together with the password to locate the payload. A wrong password or a damaged header instantly breaks the subsequent path - that is what `test_pwd_mismatch()` checks.

Seed derivation in the project (real code)

```
def derive_seed(image_bytes, password=""):
    base = hashlib.sha256(image_bytes).hexdigest()
    if password:
        base = hashlib.sha256(
            base.encode() + password.encode()).hexdigest()
    return int(base, 16)
```

6.3 Deterministic Shuffle: splitmix64 + Fisher-Yates

A seed alone is not enough; it must become "a deterministic permutation of 1..N". The project uses splitmix64 as the pseudo-random source and runs Fisher-Yates: from the end forward, each step

swaps with a position chosen by a pseudo-random number. The same seed always yields the same permutation - that is the mathematical basis of ”self-synchronization”.

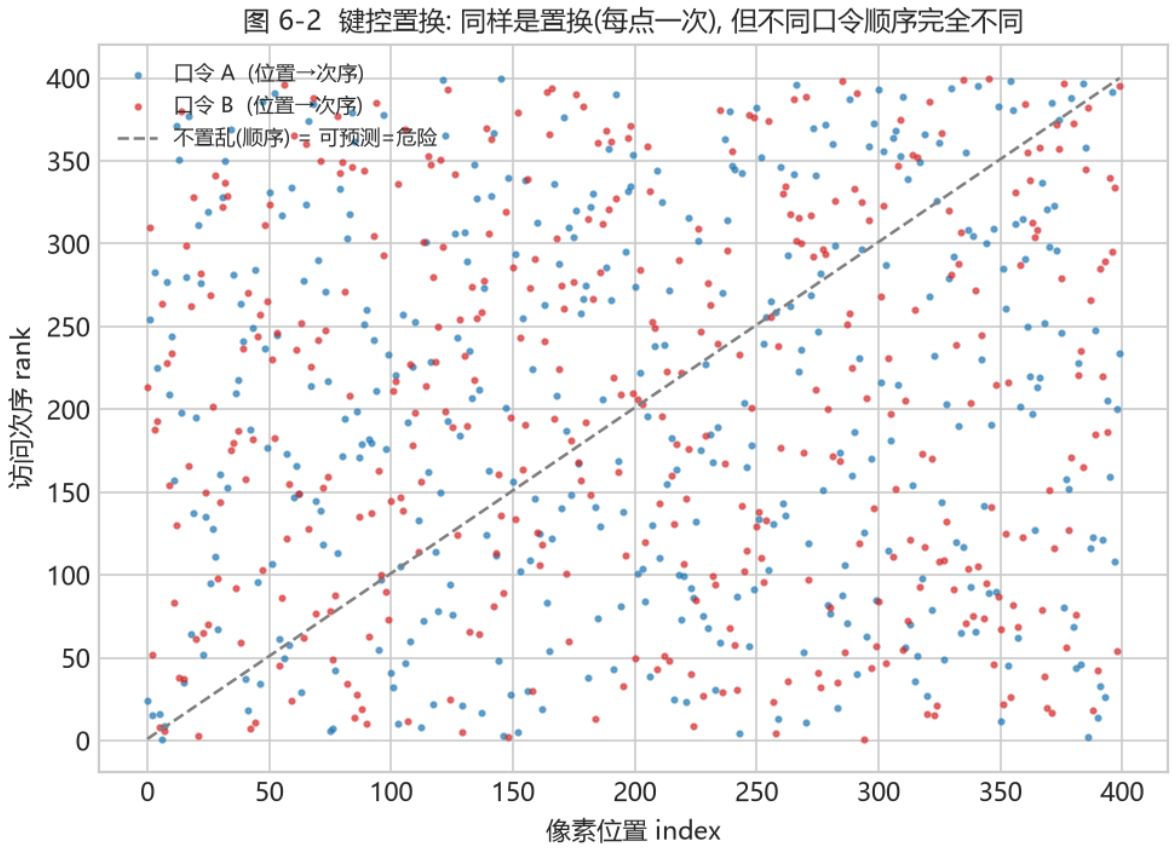

图 15: Fig. 6-2 keyed permutation scatter

Fig. 6-2 (real computation: x-axis = pixel position, y-axis = visit rank. Password A (blue) and password B (red) are each a permutation that uses every point once; the grey dashed line is the no-shuffle order (directly predictable = dangerous). The same password is always the same curve -> self-sync; a different password scatters it entirely)

Back to the code

Open `permute_index()` in `src/ns5_core.py`: when the DLL is available it calls C++ `nsf5_permute`, otherwise it uses the same-algorithm Python fallback. **Both implementations must give exactly the same permutation**, otherwise an image embedded with the DLL cannot be decoded on a DLL-less machine. `src/cppembed.py`'s `selfcheck()` does exactly this cross-language consistency check.

6.4 Tamper Perception: What It Can Actually Sense

The embedder reports `cover_hash` (the original image hash) in its report. If any pixel changes, the image bytes' SHA-256 changes. Recomputing the hash of the received image and comparing it

with the reported (or header-decoded) hash: mismatch = the image was altered. Better still, since the body path depends on this hash, even a one-pixel change shifts the body shuffle seed, and decoding immediately breaks or turns into garbage.

Be honest about the limits: this is **keying and tamper perception**, not cryptographic **integrity authentication**. The header hash is not MAC'd with a key; in theory an attacker able to re-embed the whole image can also update the header hash. The paper and README list this as future work (e.g. a keyed MAC), so distinguish "detecting ordinary tampering" from "resisting malicious re-embedding".

Tip

One sentence distinction: **tamper perception** = "I see it was altered"; **integrity authentication** = "I can prove it was not altered and an attacker cannot cover it up". The former detects, the latter resists - different strengths.

6.5 Hands-On: Wrong Password and Pixel Tampering

Experiment A: a mismatched password should fail; Experiment B: decode after changing 1 pixel

```
import sys; sys.path.insert(0, "src")
import numpy as np; import image_io as IO
from ns5_core import embed_string, extract_string, get_image_hash

img = IO.load_as_gray("img/cover.png")
stego, rep, _ = embed_string(img, "secret", method="nsF5",
                             p=3, password="A")
print(extract_string(stego, method="nsF5", p=3, password="B"))
tampered = stego.copy(); tampered[0, 0] ^= 1   # change only 1 pixel
print(get_image_hash(stego)[:16], "vs", get_image_hash(tampered)[:16])
print(extract_string(tampered, method="nsF5", p=3, password="A"))
```

Watch out

Experiment B may fail as a ValueError, garbage, or a truncated message, depending on which block the changed pixel falls in. Do not expect "always an exception" - tamper perception's engineering behavior is: the hash comparison fails and the payload cannot self-synchronize.

6.6 Summary and Self-Check

- Fixed path = predictable = can be copied and targeted; a keyed path makes the hiding positions depend on the password + content (Fig. 6-1);

- The header holds a 128-bit content hash, and the body seed depends on it -> decoding needs no external sync;
- A deterministic permutation (splitmix64 + Fisher-Yates) is the mathematical guarantee of self-sync (Fig. 6-2);
- Keying is not integrity authentication; strong adversarial scenarios need a keyed MAC.

Think about it

If two different original images are used to embed the same message, will the hidden positions in the two stego images be the same? Why? (Hint: what determines the body seed?) One layer deeper: since security relies on ”not knowing the position”, could an attacker ”guess” the shuffle rule via statistics? (See Chapter 8 - that is exactly detection’s entry point.)

Chapter 7 - Machine Learning Foundations (First Time Learners, Weeks 7-12)

Try it

Goals: fully and solidly learn four things - **how an image becomes numbers (features) -> how a model learns to judge -> how to prevent cheating -> how to tell if a model is good.** We start from a point of view that someone who has never touched machine learning can follow, with real-data figures, progressing step by step without rushing.

7.0 Chapter Roadmap: How to Study (about 3-6 weeks)

If you have a 6-12 month cycle, treat this chapter as a **real first course in machine learning**, not a crash course to "understand the project code." Suggested path:

1. **Step 1 (1 week)**: read only 7.1-7.2, build the "data -> features -> labels" picture, and be able to name everyday examples of "describing an image with numbers".

2. **Step 2 (1 week)**: read 7.3-7.5 with Fig. 7-1/7-2/7-3, understand "draw a line + turn the line into probability + make the probability right". **No formula derivation required**, just be able to retell the idea by looking at the figures.

3. **Step 3 (1 week)**: read 7.6-7.8, focus on **data leakage** and **how to evaluate**. This is where real projects most often stumble, and it deserves extra time.

4. **Step 4 (1 week)**: run every code excerpt on your own machine (`python src\train_model.py`) and confirm where each output number comes from.

How to study

Every section ends with a "Think about it". Write the answer down rather than just thinking in your head - writing is where real learning happens.

—

7.1 Steganalysis Is Just ”Is This Image Hiding Something?”

Change perspective: a photo is either clean (marked 0) or had a message hidden in it (marked 1). Machine learning just learns a judgment: give it an image and try to guess 0 or 1.

This is like a teacher giving students many problems with answers, and then students solve unseen problems on a test - machine learning does not ”memorize”, it learns **the rule**. Supervised learning has four pieces: **data, features, model, evaluation**. We go through each, mapping every one to project code.

Mini-glossary

You will see these four words throughout. First remember their ”plain” version: - **Data**: a pile of ”examples with answers”. - **Feature**: the short list of numbers an image is compressed into (like ”measuring the image with rulers”). - **Model**: a ”features-to-answer” judgment rule. - **Evaluation**: judging whether that rule is reliable.

7.2 How an Image Becomes Numbers: Features and Labels

A computer does not ”see” images, only numbers. So step one is to **compress an image into a short list of numbers**. The project's v1 compresses each image into **11 numbers**, and v1.4 extends this to 143 (Chapter 8). Those 11 numbers are the **feature vector** - like ”measuring the image with 11 rulers.”

- **Feature**: the 11 numbers (e.g., ”how chaotic the pixel differences are”, ”is the brightness distribution off”).
- **Label**: whether the image hid something; 0 = clean, 1 = hidden.
- **Sample**: one row = one image, 11 feature numbers + 1 label.

What the dataset looks like (first columns are features, `label` is the answer, ignore `photo_id` for now)

```
Rm,Sm,Rn,Sn,RS_Gr,RS_Gn,chi2_pvalue,diff_entropy,lsb_diff_entropy,median_prefix_p
    ,chi2_stat,label,photo_id,...
41316,4574,36608,5057,0.56,0.48,0.56,1.99,0.986,2.7e-166,1719.3,0,0,...
40488,4907,35701,5342,0.54,0.46,0.61,2.02,0.989,2.2e-155,1675.9,1,0,...
```

data/dataset.csv (real first two rows, numbers truncated for display)

Tip

Do not panic at names like `chi2_pvalue` or `RS_Gn`. They are just ”the number from ruler number N.” Chapter 8 explains what each ruler measures.

Write the table as notation - every formula below uses it (do not fear; it is just ”row i, column j”):

- **Sample**: one row is one sample, N in total, indexed $i = 1, \ldots, N$;
- **Features**: the 11 numbers of sample i are a vector $\mathbf{x}_i$; all rows form a matrix X (N rows x 11 columns);
- **Label**: $y_i \in \{0, 1\}$, stacked into a vector $\mathbf{y}$.

7.3 Classification = Draw a Line to Separate Two Groups

Once we have a pile of ”numbers with answers” (features + labels), what does the model learn? It learns **to draw a line in number-space: one side clean, one side hiding.** Imagine: horizontal axis = ”how chaotic”, vertical axis = ”is brightness balanced”. Clean images cluster in one corner, hiding images in the other. The model learns that dividing line.

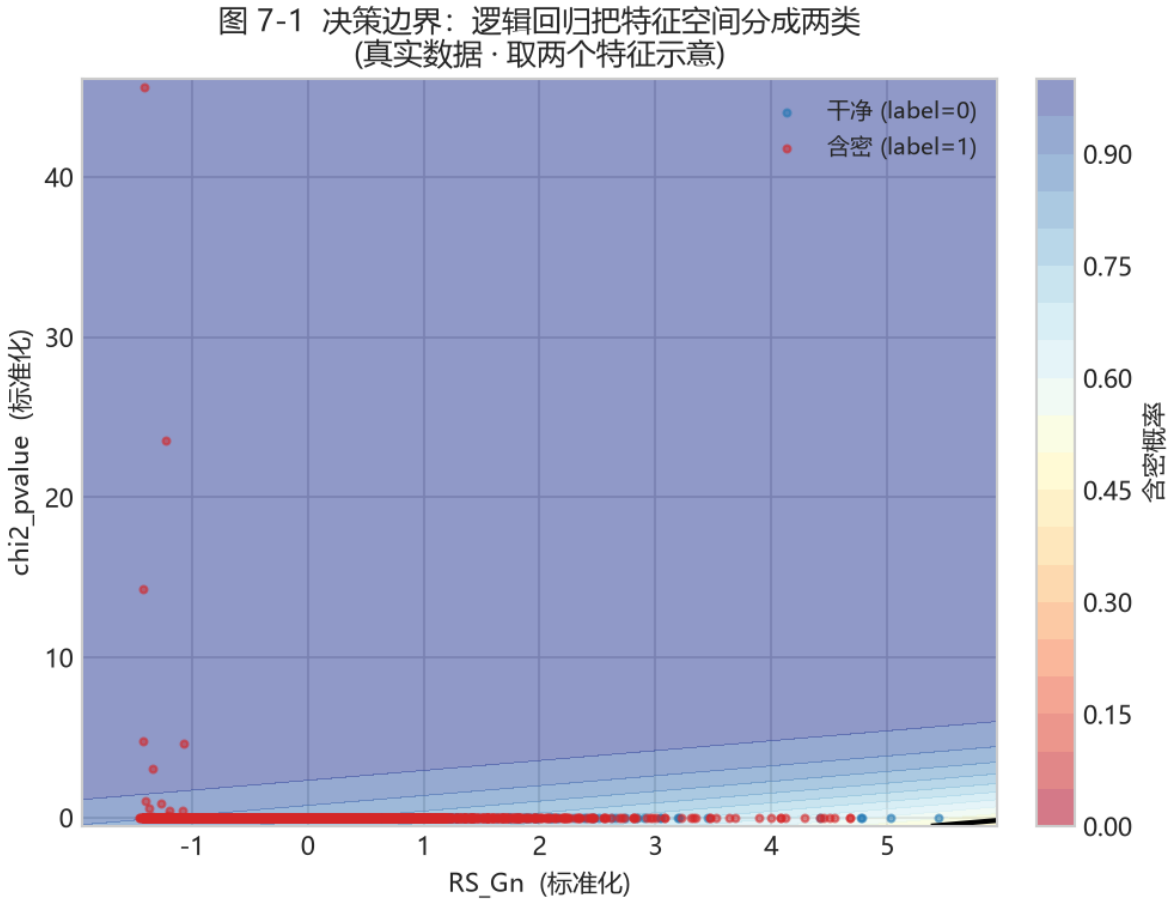

图 16: Fig. 7-1 decision boundary

Fig. 7-1 (real data: decision boundary learned by logistic regression on the RS_Gn and chi2_pvalue features; blue = clean, red = hiding, shading = model's estimated hiding probability)

Understanding this figure is the key to the whole chapter. Notice:

1. **The two classes are not perfectly separable**: near the boundary blue and red points mix. The two features are not enough to fully separate, but already "roughly" separate.
2. **The shading is "probability"**: the redder, the more the model thinks hiding; the bluer, the more clean. The black line in the middle is the "probability = 0.5" boundary.
3. **More dimensions work the same**: the real model uses 11 (or 143) features - it draws a "hyperplane" in 11 (or 143) dimensions. You cannot draw that, but the idea is identical to this 2-D figure.

In math this line is called a **linear discriminant function** - basically a scoring formula:

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_{11} x_{11} + b.$$

Do not let the symbols scare you. It is a weighted sum:

- $x_1, \ldots, x_{11}$ are the 11 feature numbers;
- $w_1, \ldots, w_{11}$ are **weights** - they decide "which feature matters and which way the line tilts." If a feature is especially useful, its w is big;
- b is the **bias** - it shifts the whole line up or down to set the position.

Judging is simple: $z \geq 0$ means "hiding", $z < 0$ means "clean". **The single line `LogisticRegression` is the code that automatically finds the best w and b.**

Think about it

In Fig. 7-1, which of "RS_Gn" and "chi2_pvalue" seems more important for the judgment? If you could keep only one feature, which would you pick? - Your intuition is verified later in the "feature importance" figure in 7.7.

7.4 From Score to Probability: Why a sigmoid?

z above is a signed, unbounded "score." But the user wants a probability between 0 and 1: "how likely is this image hiding something?" So we add a **sigmoid** that squashes the score to 0-1:

$$\hat{p} = \frac{1}{1 + e^{-z}}.$$

- Very large z (high score) -> $\hat{p}$ near 1 (probably hiding);
- Very small z -> $\hat{p}$ near 0 (probably clean);

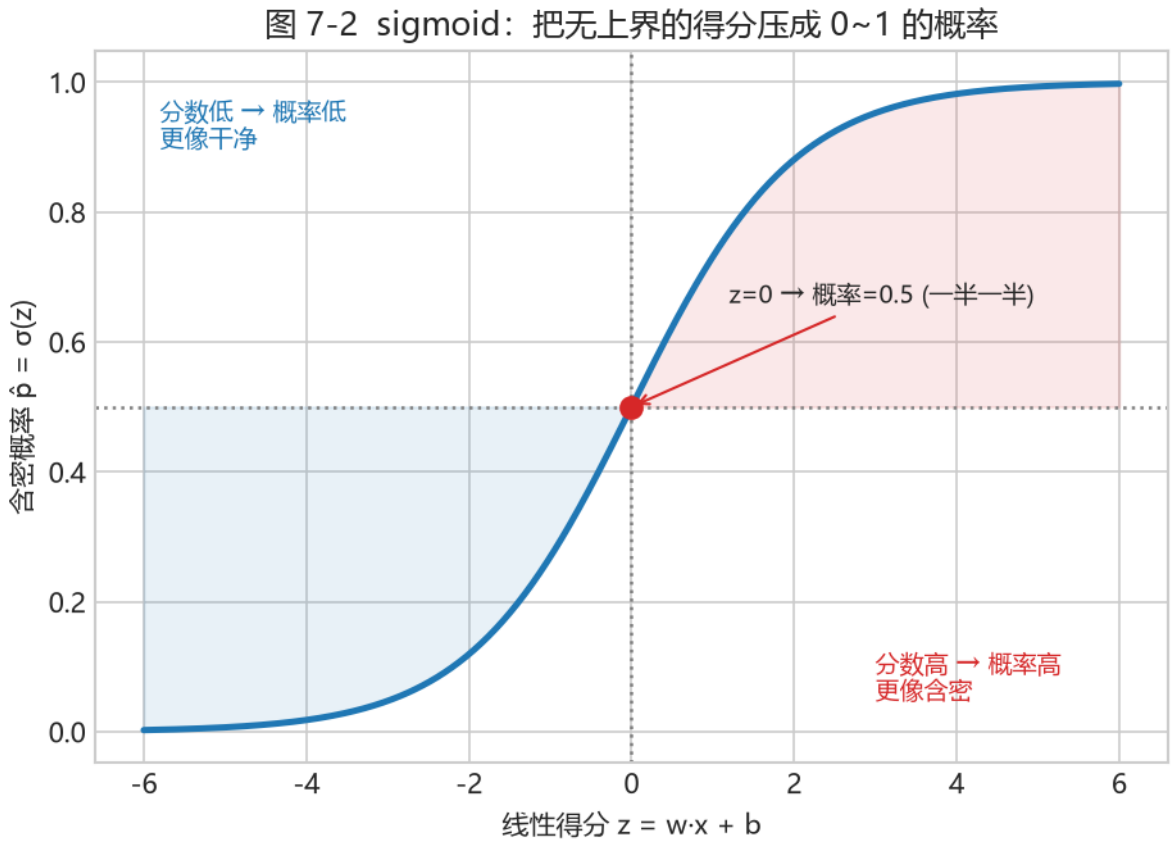

图 17: Fig. 7-2 sigmoid

- $z = 0$ -> exactly 0.5 (fifty-fifty).

Fig. 7-2 (sigmoid compresses the unbounded, signed "score z" into a 0-1 "probability". Left blue region = more like clean, right red region = more like hiding)

Tip

Think of sigmoid as a "percentage converter": it turns a boundary-less score into an easy 0%-100%. **Despite the name "logistic regression", it is still "draw a line + output a probability."**

7.4.1 Why Not Just Output 0/1, but a Probability?

Good question. Because the real world is not black-and-white. In Fig. 7-1, the points near the boundary are genuinely uncertain. Rather than forcing the model to say "0" or "1", it is honest to say "I am 73% confident it is hiding." Then:

- The user can choose the **threshold** themselves (7.7);
- You can evaluate **how confident** the model really is (calibration);
- You can draw threshold-independent metrics like ROC / AUC (7.7.1).

Mini-glossary

A classifier that outputs probabilities is **soft classification**; one that directly outputs 0/1 is **hard classification**. Logistic regression is soft - that is why it can compute AUC.

Against the project code - `src/train_model.py`.

```
def lr(seed=0):  return make_pipeline(StandardScaler(), LogisticRegression(
    max_iter=2000, C=0.1, random_state=seed))
```

Code	Plain explanation
`StandardScaler()`	First make every feature "about the same size", so a huge number does not skew the line (see 7.4.2)
`LogisticRegression`	The "draw a line + sigmoid output probability" above; finds w, b automatically
`max_iter=2000`	Adjust at most 2000 times, so it does not stop early
`C=0.1`	A dial that says "do not memorise the answers too hard" (regularization, 7.5.2)

If you want to go deeper

Why standardize? Because the chi-square statistic might be in the thousands while Gn is only 0-1. If the weights treat both equally, the gradient spends all its effort pleasing the big number and the line gets skewed. Standardization is `x'=(x-`μ`)/`σ, turning every feature into "mean 0, standard deviation 1", so everyone is fair. Note: μ, σ must be computed on the training set only; never sneak the test set in.

7.5 How a Model "Learns": Loss and Training

What does "learning" actually do? It **keeps adjusting w and b so the output probability gets closer to the correct answer.** How to measure "close"? Use a **loss** - think of it as "points deducted for being wrong." The classic is cross-entropy (log loss):

$$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\Big[y_i \log \hat{p}_i + (1-y_i)\log(1-\hat{p}_i)\Big].$$

In plain words: ***when the true answer is 1 (hidden), only $\hat{p}_i$ matters - the closer to 1, the fewer points lost; when the true answer is 0, only $1-\hat{p}_i$ matters - the closer to 0, the fewer points lost.*** *The more wrong, the more you lose.*

So how to lose fewer points? **Try a little bit at a time**: compute "which tiny adjustment lowers the loss fastest" and step that way. That is **gradient descent**:

$$\theta \leftarrow \theta - \eta \frac{\partial \mathcal{L}}{\partial \theta}.$$

Fig. 7-3 (a simple "loss vs parameter" curve. The red staircase is gradient descent: start from the left and step down to the valley - the w^* where loss is smallest. The blue curve is the loss function, and the valley is the optimal parameter)

Tip

You absolutely do not need to derive the derivative. Remember three sentences: **loss = how wrong the model is; training = find parameters that minimize loss; gradient descent = keep nudging in the direction that lowers loss fastest.** scikit-learn wraps all this up for you.

7.5.1 Why Go Step by Step Instead of One Shot?

In theory logistic regression's loss is convex and has a closed-form solution. But in real projects:

- With lots of data, inverting a big matrix is slow and unstable;
- We want a method that generalizes to many models (trees, neural networks);
- Gradient descent is the common engine for **any** model that minimizes a loss.

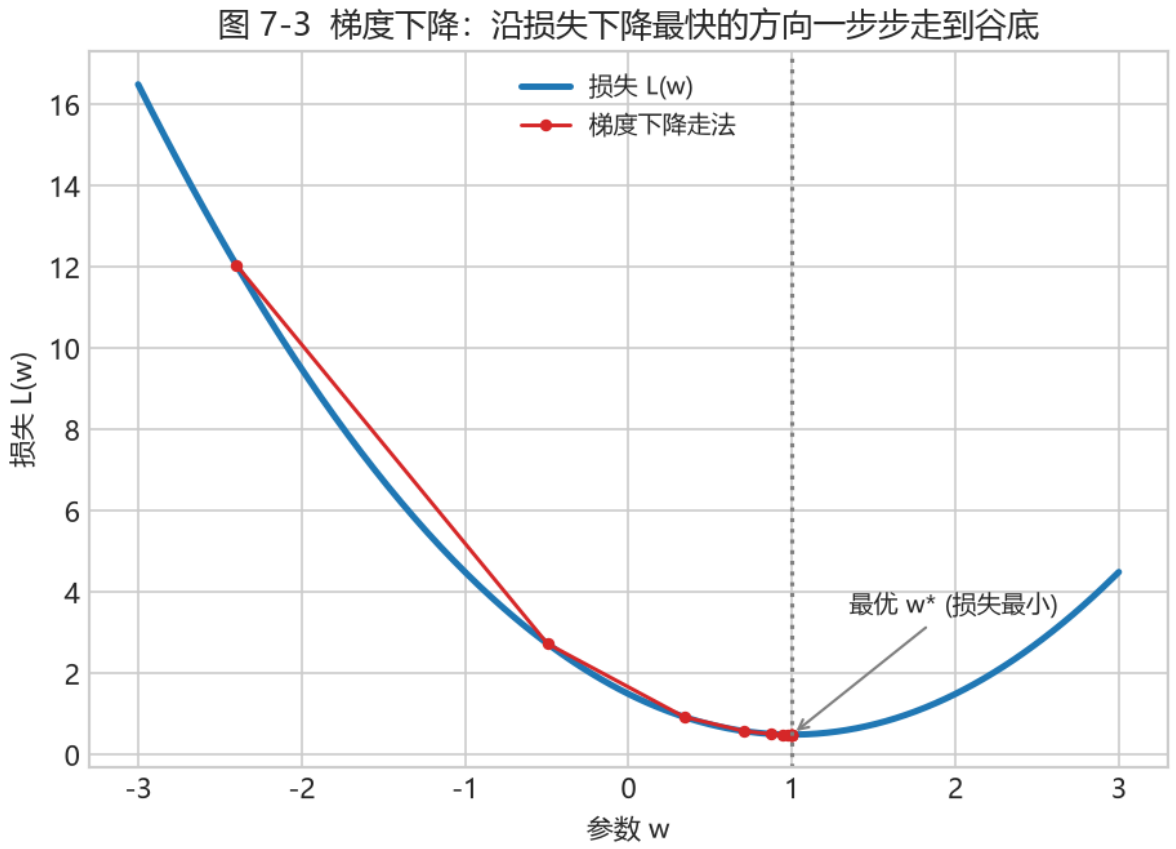

图 18: Fig. 7-3 gradient descent

So understanding gradient descent means understanding how ”training” works in linear, tree, and even neural models at once. That is its value.

7.5.2 An ”anti-overfitting” dial: C and regularization

If features are highly correlated or samples are few, the model may inflate weights to memorize the training data - a sign of **overfitting**. One defense is to add a penalty encouraging small weights (L2):

$$\mathcal{L}_{\text{reg}}(\mathbf{w}) = \mathcal{L}(\mathbf{w}) + \lambda \sum_j w_j^2.$$

scikit-learn uses the **inverse regularization** $C = 1/\lambda$:

- `C=0.1` -> $\lambda = 10$, strong regularization, weights sharply compressed, conservative;
- `C=1.0` -> $\lambda = 1$, moderate; larger `C` means less regularization.

Back to the code

`train_model.py` uses `LogisticRegression(C=0.1)` for base models (want them restrained, not memorizing), while the stacking meta-learner uses `C=1.0` (looser, freer to combine). Same class, two C - the trade-off ”base models steady, meta-learner flexible.”

7.6 How Not to ”Cheat”: Cross-Validation and Grouping

Getting a high score on seen problems is easy; the real question is **unseen problems**. If a model memorizes the training samples, it collapses on new ones - **overfitting**.

The standard defense is **cross-validation**: cut the data into pieces and rotate which piece is the ”mini-test”. But there is a trap easy to step into.

Watch out

One photo has 7 variants (1 clean + 6 hidden). They are **too similar**. If you split randomly, training could contain a ”blood sibling” of some test photo - effectively telling the model the answer early. That is **data leakage**, and the score balloons unrealistically. The project’s fix: **group by `photo_id`** - every variant of a photo is entirely in training or entirely in test, never split.

Fig. 7-4 (schematic: each P is a photo, one row has 7 variants. Top ”random split” tears the 7 variants of a photo into different folds - leaks. Bottom ”GroupKFold” puts all 7 variants of a photo into the same fold - honest)

Against the code - the core of `src/train_model.py::cv_oof()`.

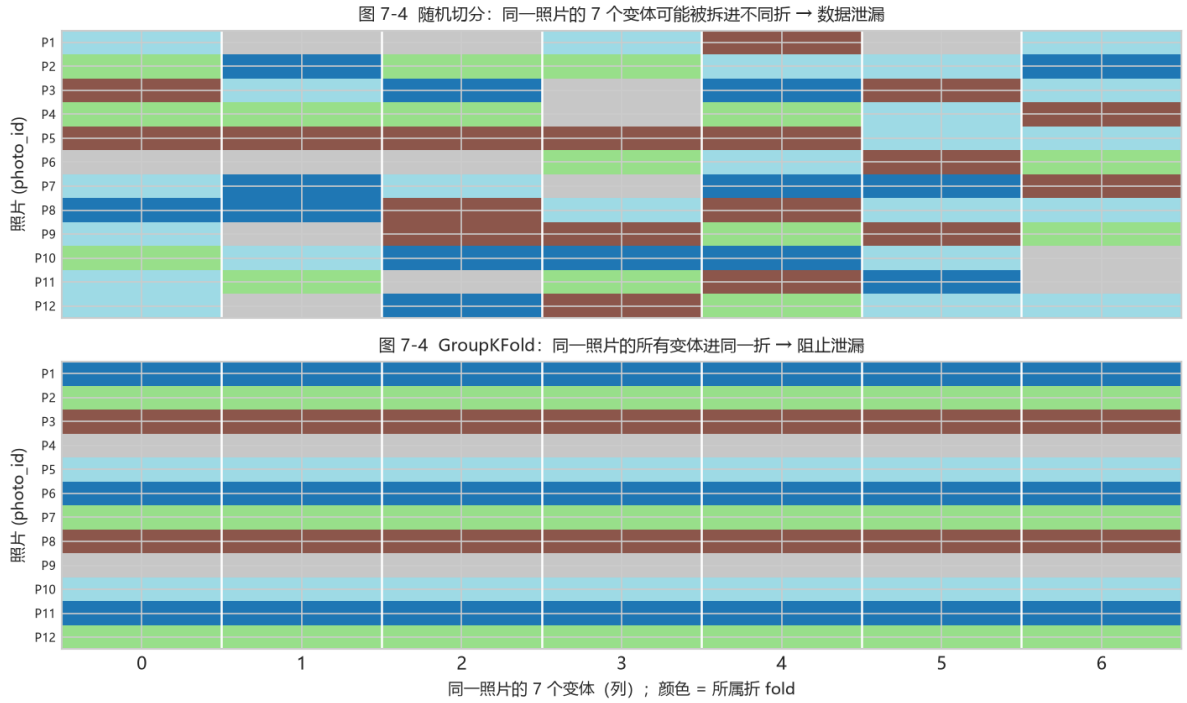

图 19: Fig. 7-4 GroupKFold

```
from sklearn.model_selection import GroupKFold
gkf = GroupKFold(n_splits=5)
for tr, va in gkf.split(X, y, groups):    # groups = photo_id, split by photo
    clf.fit(X[tr], y[tr])
    p = clf.predict_proba(X[va])[:, 1]    # score only on this "mini-test" fold
```

The key is the `groups` argument. It tells the splitter: ”samples in the same group must not be torn apart.” The ordinary `KFold` would split a photo’s ”blood siblings” across both sides, and the score would cheat.

Think about it

Why do the 7 variants of one photo not count as independent samples? Try a minimal example: with only 10 photos, how likely is a random split to put some photo’s ”blood sibling” on both training and test sides? How much does such ”leakage” inflate the score? (Hint: think about the 0.5 random line in 7.7.1)

7.7 Evaluation: Do Not Just Look at Accuracy

The model outputs a probability, so we pick a **threshold** to say 0 or 1. For evaluating, start with the **confusion matrix**:

Fig. 7-5 (real data at the Youden threshold: TN = correct reject, FP = false positive, FN = miss,

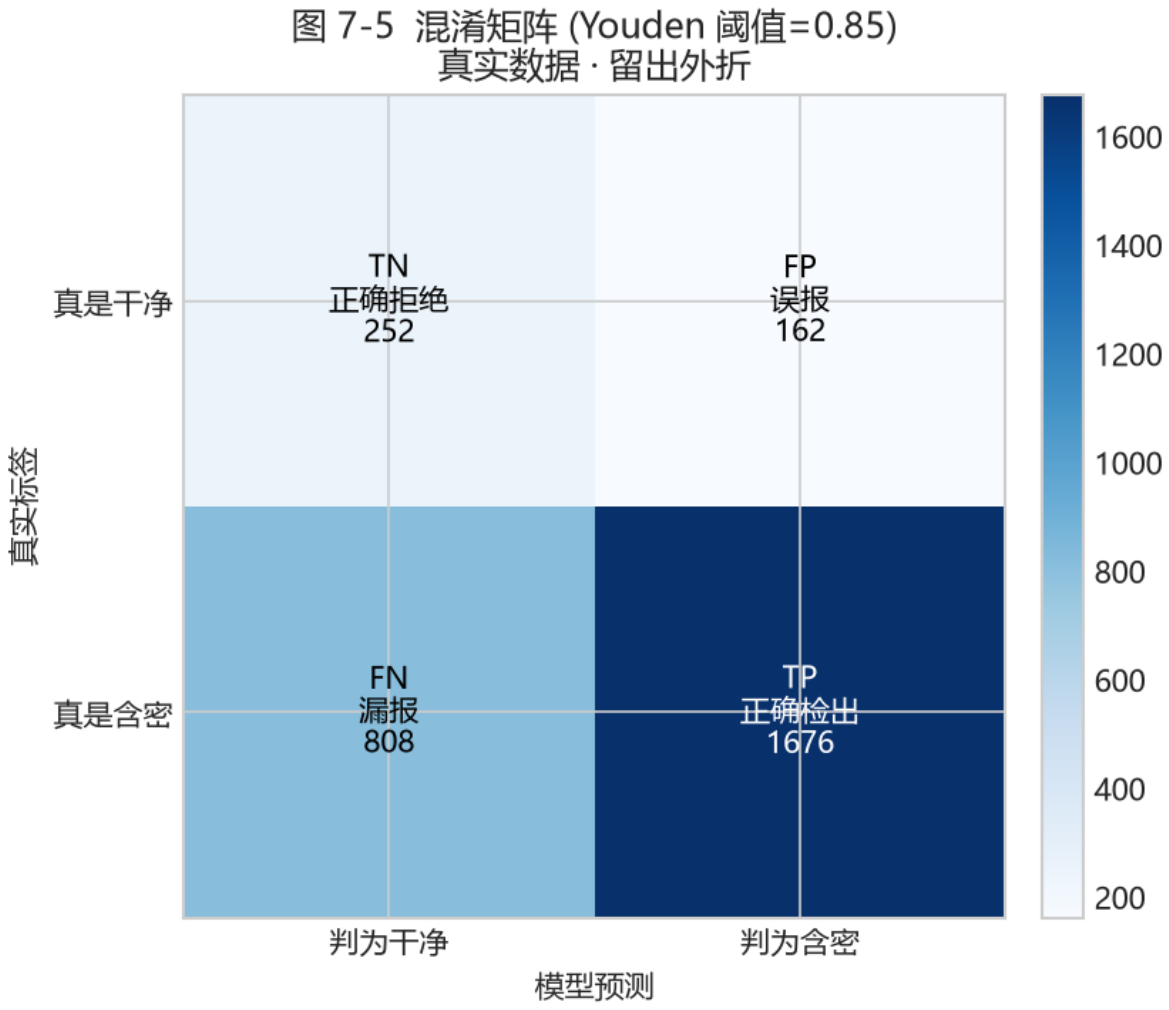

图 20: Fig. 7-5 confusion matrix

TP = correct detection. Numbers are real sample counts)

From these four values (TN, FP, FN, TP), the common metrics:

$$\text{accuracy} = \frac{TN+TP}{TN+FP+FN+TP}, \quad \text{precision} = \frac{TP}{TP+FP}, \quad \text{recall} = \frac{TP}{TP+FN}, \quad F_1 = \frac{2 \cdot precision \cdot recall}{precision + recall}.$$

- **Accuracy**: fraction correct overall. **Trap**: if there are far more hiding than clean, a model labeling everything "hiding" still scores high accuracy - but falsely accuses every clean image.
- **Precision**: of those flagged hiding, how many really are.
- **Recall**: of the truly hiding images, how many were caught.
- **F1**: a compromise between precision and recall.

Metric	Question it answers	Intuition
Accuracy	How many right overall	Misleading when imbalanced
Precision	Of flagged hiding, how many really are	Low false-positives -> high
Recall	Of truly hiding, how many caught	Low misses -> high
F1	When you want both	Compromise
ROC / AUC	Ranking strength, threshold-free	0.5 = random

7.7.1 ROC Curve and AUC: Can You Rank Hiding Above Clean?

The key idea: a good model should rank hiding images above clean ones. Sweep the threshold from high to low and you get an **ROC curve**, horizontal = "false-positive rate", vertical = "detection rate". The closer to the top-left corner, the better.

Fig. 7-6 (real-data OOF ROC curve: AUC 0.70. The red shaded area is the AUC - "draw one hiding and one clean image at random; the probability the model ranks the hiding image higher." The red dot is the Youden best operating point)

AUC is the area under the curve: 0.5 is random (diagonal dashed line), 1.0 perfect ranking. **0.7 means "usually ranks hiding above clean, but not great"** - matching the reality that weak densities are hard to detect.

Key point

AUC is threshold-independent! No matter whether you set the threshold to 0.5 or 0.7, AUC stays the same. Because AUC only cares about **ranking**, not "where exactly to cut." You can see this directly in Fig. 7-7.

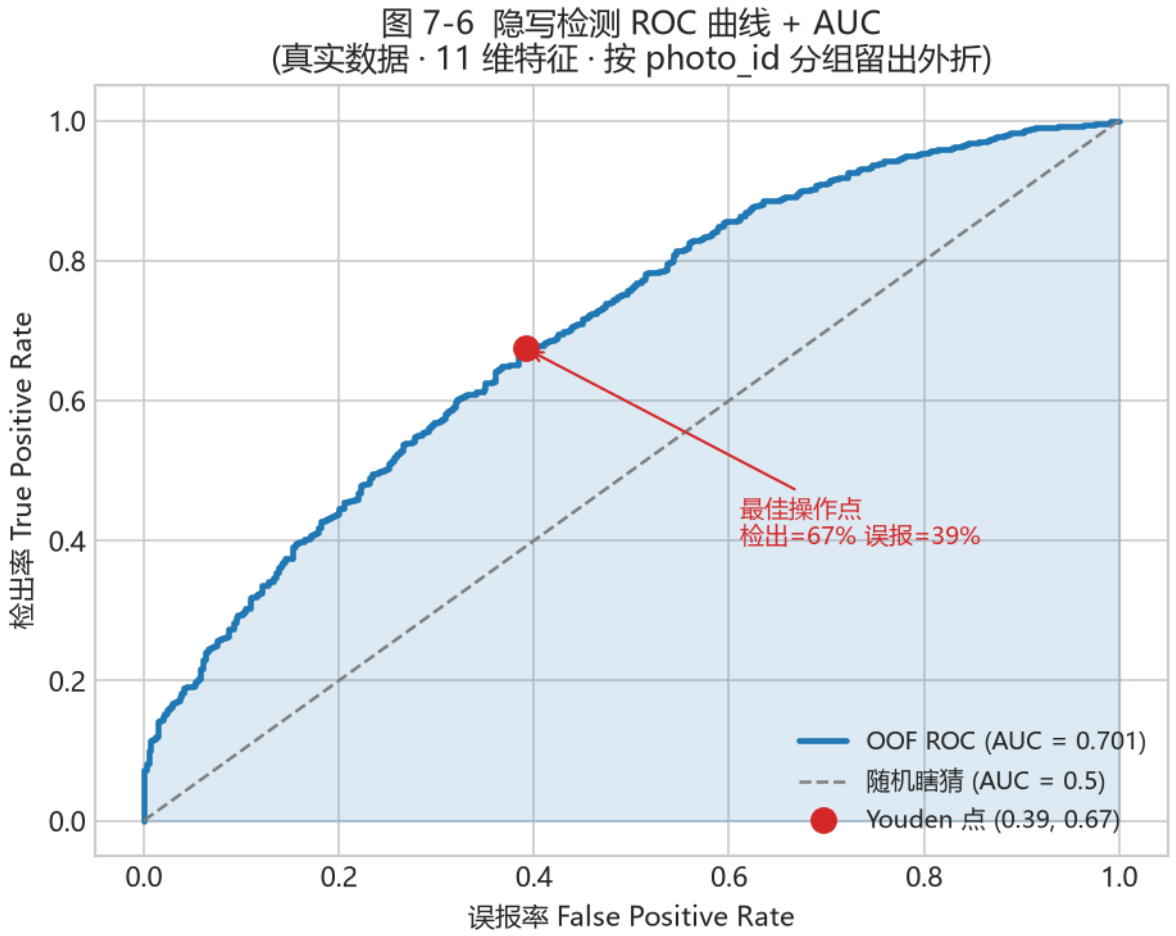

图 21: Fig. 7-6 ROC + AUC

7.7.2 Choosing a Threshold: The "Operating Point" on the ROC

Although AUC is threshold-independent, **in real deployment** you must choose one. Higher -> conservative (fewer false positives, more misses); lower -> aggressive (more detection, more false positives). Look at this figure:

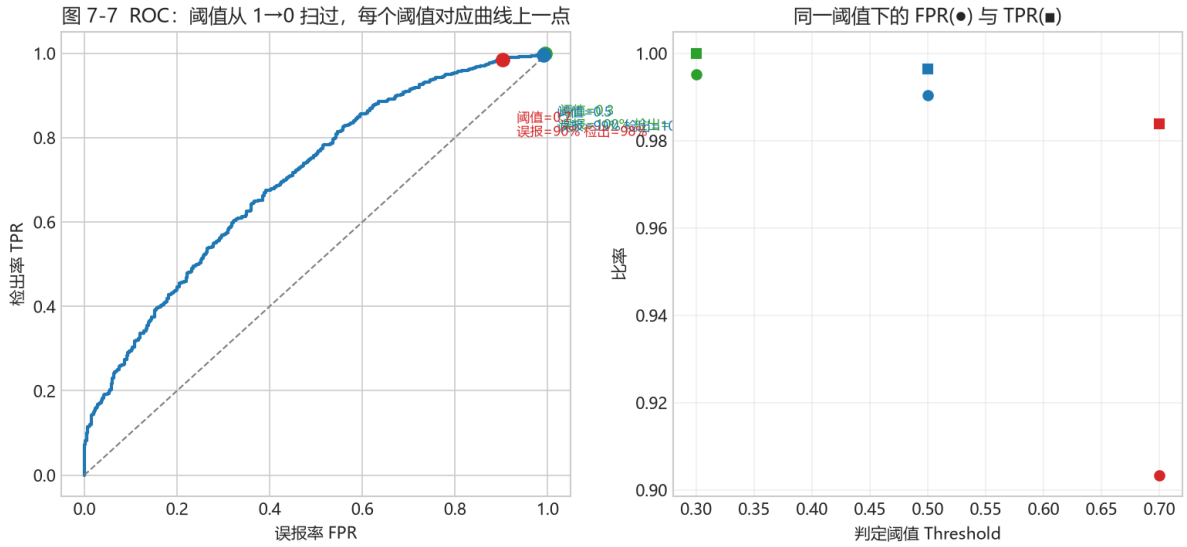

图 22: Fig. 7-7 ROC operating points

Fig. 7-7 (same test set. Left: thresholds 0.3/0.5/0.7 as three points on the ROC; Right: same thresholds' detection rate TPR( ) and false-positive rate FPR( ). The lower the threshold, the higher the detection but also the higher the false positives - an unavoidable pair)

Back to the code

`train_model.py` picks the threshold with the Youden rule - the point maximizing "detection minus false-positive" (farthest from the random line), and also computes a low-FP threshold (FP<=10%) for strict mode:

```
from sklearn.metrics import roc_curve
fpr, tpr, th = roc_curve(y_true, proba)
j = tpr - fpr                # Youden J = detection - false-positive
best = float(th[np.argmax(j)])   # threshold with largest J, farthest from random
    line
```

7.7.3 Why "Accuracy" Misleads Here

Imagine 2,484 of 2,898 samples are hiding: a model labeling **everything hiding** reaches accuracy 2484/2898 $\approx$ 85.7%, which looks great but **false-positives all 414 clean images**. So accuracy is meaningless under class imbalance; look at AUC, recall, and F1 instead.

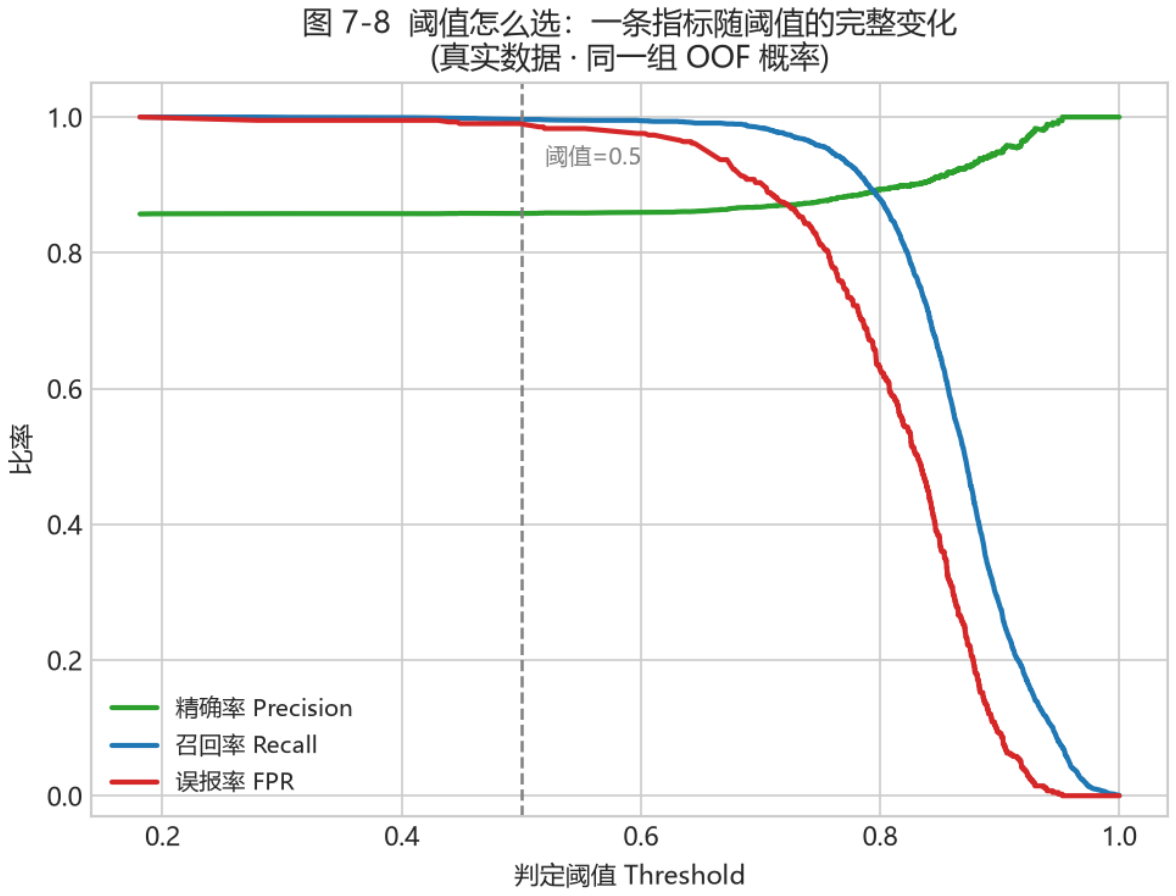

图 23: Fig. 7-8 metrics vs threshold

I have drawn a figure showing exactly how each metric changes with the threshold:

Fig. 7-8 (real data: as threshold falls from 1 to 0, precision (green), recall (blue), false-positive rate (red) change. Your chosen threshold is an "operating point" - a trade-off here)

Reading the figure:

- Very low threshold: high recall (catch almost all), but low precision, high false-positive rate (over-flag clean images);
- Very high threshold: high precision (flagged ones mostly real), but low recall (miss many);
- Somewhere in between, you pick a "compromise point" that fits your need.

Try it

Run `python src\train_model.py` once. You do not need to understand all output; first find three lines: CV-AUC, held-out test AUC, and low-FP detection rate. Chapter 8 explains them line by line.

7.8 Advanced: Two Bonus Bits in the Project (good to know roughly)

`train_model.py` does two more things. On first read, just get the gist:

- **Probability calibration**: logistic regression is often overly confident (says 0.95 when it should be 0.3). The project adds calibration so probabilities match real proportions, making thresholds meaningful.
- **Stacking**: it does not bet on one model. It takes the answers of 4 models (logistic regression, random forest, gradient boosting, XGBoost) and lets a small model (logistic regression) learn "whose advice to follow".

Tip

Stacking is like a jury: each juror (base model) scores independently, then a chairperson (meta-learner) combines the verdict. It does not pick one best; it merges the strengths of several ideas. Fig. 8-4 in Chapter 8 compares the four models side by side - then you see why they can "complement" each other.

7.9 Summary and Self-Check

One sentence: machine learning = use "data -> features -> model -> evaluation" to turn "did this image hide something" into a judgeable, evaluable, explainable process.

- Supervised learning = use "samples with answers" to learn a features-to-answer judgment;

- Classification = separate two groups with a boundary; logistic regression = draw a line + sigmoid to probability (Fig. 7-1, 7-2);
- Loss = points lost for wrong guesses; training = minimize loss; gradient descent = step downhill a little at a time (Fig. 7-3);
- Do not randomly split same-source samples; `GroupKFold` grouping by `photo_id` is the honest way (Fig. 7-4);
- Accuracy distorts under imbalance; look at AUC / precision / recall; the threshold decides ”strict or loose” (Fig. 7-5 7-8).

Think about it

A paper says ”steganalysis AUC = 0.95” but never says how the data were split. What do you suspect first? Write it down - that is the start of evaluating every ML experiment. Go deeper: did it do calibration? How was its threshold chosen? Do these details also quietly cherry-pick a nice result?

Try it

After understanding Fig. 7-6 and 7-8, answer: if the boss demands ”false positives must stay below 5%”, should you raise or lower the threshold? What is the cost?

Chapter 8 - Machine-Learning Steganalysis (Weeks 8-14)

Try it

Goals: understand **why features instead of raw pixels**; recognize what each of the 11 features measures (with real distribution figures); read 4 classifiers and 4 real ROC curves; run the full "generate data -> train -> judge" flow. Section 8.8 covers the complete v1.4 evolution.

8.0 Chapter Roadmap (about 4-8 weeks)

1. **Step 1 (1 week)**: 8.1, understand "why features instead of a raw CNN" - the **most important step** in this book; make sure you get it.
2. **Step 2 (2 weeks)**: 8.2, meet the 11 features one by one. Use Fig. 8-1/8-2 distributions, cross-check formulas and `steganalysis.py`.
3. **Step 3 (2 weeks)**: 8.3-8.4, understand "where data come from + how to read the training script". Use Fig. 8-3/8-4.
4. **Step 4 (2 weeks)**: 8.5-8.7, learn to read ROC / detection rates and run the full pipeline.
5. **Step 5 (advanced, 1 week)**: 8.8 v1.4 evolution (SRM, 143-D, dual models) - the part closest to a paper.

—

8.1 A Counterintuitive Question: Why Not Just Let the AI "Look"?

You heard deep learning can recognize cats and dogs. Why not feed raw pixels to a network and let it learn "hidden or not" by itself? The project tried - **validation AUC 0.50, essentially random guessing**. The reasons:

- Steganography changes an **extremely weak high-frequency signal** (the LSB layer), far weaker than image content or texture;

- A network easily learns ”this is a landscape, that is a portrait” rather than ”hidden or not”;
- The number of truly independent samples is the **number of photos** (hundreds), while deep models usually need thousands.

So the project took another path: **use human domain knowledge to measure the statistical traces hiding leaves (features), then let a simple classifier judge.** v1 used 11 features with AUC 0.75-0.79 on small data; v1.4 raised it to 0.90+ with 143-D features and LightGBM (see 8.8), but ”features beat raw CNN” still holds.

Tip

This is not ”deep learning is useless”, but a lesson in **how to choose a method when data are scarce**: first verify the signal with strong explainable features, then consider a more complex model. Many engineering problems follow the same order.

8.2 Meet the 11 Features: What Are They Measuring?

The v1 features come from the C++ library `cpp/fsfeatures.dll` (Python wrapper in `src/fsfeatures.py`); the 143-D v2 set is built in `src/featurize_v2.py`. First, one big impression: **these 11 numbers are 11 rulers, each measuring ”was the LSB plane messed with”.** They fall into three groups:

Feature	Measures	After hiding
Rm / Sm / Gr	Regular/singular counts and gap under +M	Gr drops
Rn / Sn / Gn	Counts and gap under -M	**Gn collapses (core signal)**
chi2_stat / chi2_pvalue	Were gray pairs ”flattened”?	Pairs balance, p rises
diff_entropy	Native texture roughness	Used to spot naturally noisy images
lsb_diff_entropy	Is the LSB plane chaotic?	Approaches 1 when randomized
median_prefix_p	Median chi-square p over 20 prefixes	Rises when globally randomized

Now the ”skill” of each ruler. **Look at the distribution figures first, then the formulas** - figures are more intuitive.

8.2.1 RS Analysis: Where the Core Signal Gn Comes From

RS (Regular-Singular) groups pixels **4 at a time** $g = (g_1, g_2, g_3, g_4)$ and defines ”how smooth this group is” using the sum of adjacent differences:

$$f(g) = |g_1 - g_2| + |g_2 - g_3| + |g_3 - g_4|.$$

Pick a mask $M = (0, 1, 1, 0)$, flip LSBs at certain positions to get a new smoothness f_M; its complement is the negative mask. Each group is then:

- **Regular**: after flip $f_M > f$ (smoother/regular);
- **Singular**: after flip $f_M < f$ (messier).

Then the **gap** is the difference of the two counts as a fraction of the total:

$$G_r = \frac{R_M - S_M}{n}, \qquad G_n = \frac{R_{-M} - S_{-M}}{n}.$$

图 8-1 RS 缺口特征分布：干净 vs 含密

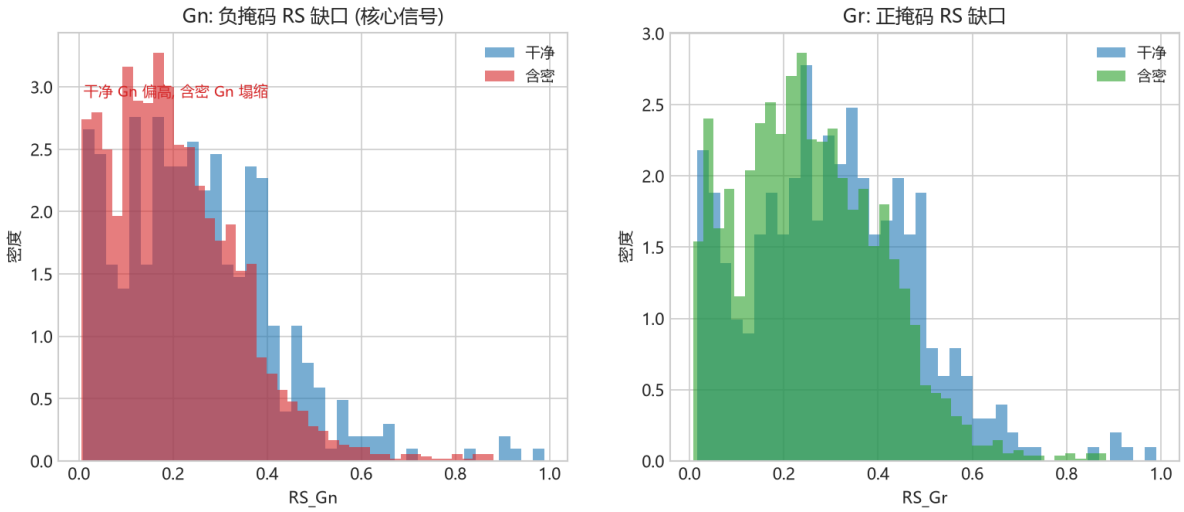

图 24: Fig. 8-1 RS feature distributions

Fig. 8-1 (real data: RS gap distributions for clean (blue) and hiding (red). Left: Gn (negative mask) - clean images cluster higher while hiding collapses toward low values; that is the strongest signal. Right: Gr (positive mask) also separates, but less cleanly than Gn)

*In plain words**: a clean image's LSB plane has structure (adjacent LSBs often correlate). Flip the negative mask and many groups go from regular to singular, so G_n is clearly positive (0.3-0.7 on clean images).** After hiding, the LSB plane is randomized, the two masks behave alike, and G_n collapses toward 0. So Gn collapse = strong evidence of hiding.*

Against the code - `src/steganalysis.py::rs_metrics`.

```
gi = groups.astype(np.int16)                          # group of 4 pixels
fg = np.abs(np.diff(gi, axis=1)).sum(axis=1)          # f(g) = Σ|adjacent diff|
pos = ((gi ^ mask) & 0xFF)                             # group after positive-mask
    flip
fM  = np.abs(np.diff(pos, axis=1)).sum(axis=1)
Rm = int(np.sum(fM > fg)); Sm = int(np.sum(fM < fg))
# finally {"Gr": (Rm - Sm) / n, "Gn": (Rn - Sn) / n}
```

8.2.2 Chi-Square Test: Were Gray Pairs "Flattened"?

Westfeld pairs gray values $(2i,\ 2i+1)$. In a clean image these two gray frequencies usually differ; LSB randomization flattens them. The **chi-square** measures deviation from uniform:

$$\chi^2 = \sum_i \frac{(O_i - E_i)^2}{E_i}, \qquad O_i = \text{observed even-gray frequency},\ E_i = \frac{f_{2i} + f_{2i+1}}{2}.$$

The p-value comes from an **incomplete gamma**: $p = Q(df/2,\ \chi^2/2)$, with degrees of freedom df = (active pairs - 1).

In plain words: a high p-value means the observation is barely different from "perfectly even" - that gray pair has **already been randomized/hidden**.

A small detail (nice to know)

: the code writes `(even-odd)²/(even+odd)`, exactly twice the textbook formula. Since it just multiplies each term by a constant, the **ranking** of "which image is more suspicious" is unchanged - only the p-value scale differs.

Against the code - `steganalysis.py::chi2_stats` and `_prefix20_p`.

```
even = counts[0::2]; odd = counts[1::2]
stat = float(np.sum((even[mask]-odd[mask])**2 / sums[mask]))    # chi-square
return stat, float(chi2_sf(stat, n-1)), n-1                     # chi-square, p,
    df
```

Back to the code

`median_prefix_p` cuts the image into 20 prefixes, computes a p-value per prefix, and takes the median - so a single segment's fluke does not sway the judgement.

8.2.3 Entropy: How Naturally Chaotic Is the Image?

The **Shannon entropy** of adjacent-pixel absolute differences measures the image's native chaos:

$$H = -\sum_{d=0}^{255} p(d)\log_2 p(d), \qquad p(d) = \frac{\#\{\text{adjacent abs diff} = d\}}{\text{total pairs}}.$$

- `diff_entropy`: entropy over grayscale differences. Smooth/structured images low (1-3), high-noise/dithered near 8.
- `lsb_diff_entropy`: keep only the LSB plane (pixel & 1), then entropy over adjacent differences, range 0-1. Fully random LSB -> 1, clearly structured -> <0.7.

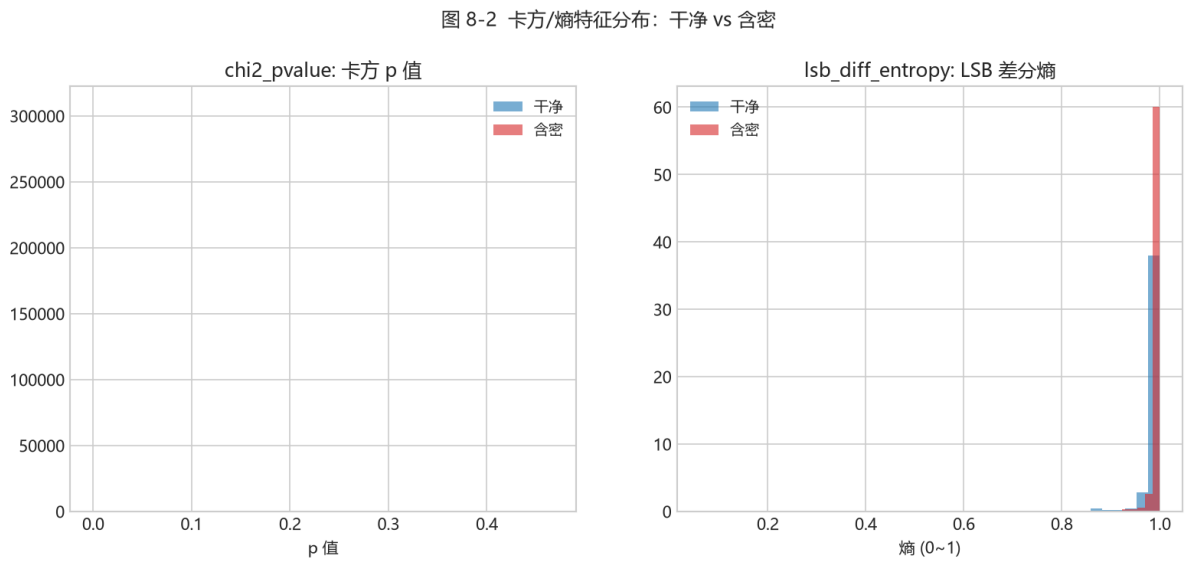

图 25: Fig. 8-2 chi-square/entropy feature distributions

Fig. 8-2 (real data: Left: chi-square p-value - hiding (red) clearly clusters near p=1 ("flattened"), clean (blue) more spread. Right: LSB difference entropy - hiding (red) toward 1 (LSB more random), clean (blue) lower)

In plain words: these two features **recognize images that are naturally messy** (high-noise photos, dithering) so we do not misjudge them as hiding - they backstop "can we even conclude reliably". For example, an image with `lsb_diff_entropy` very close to 1 could be natural noise or could be hiding; cross-check with other features.

8.3 Where Data Come From: The "1+6" Design of make_dataset.py

Training data cannot come from hand-hiding a few messages. `src/make_dataset.py` produces one clean image plus six hiding variants per source photo (v1.4 supports 12; see 8.8):

Variant (method, p, density)	Strength	Purpose
clean	-	Clean negative samples
nsF5, p=3, d=0.25	weak	Close to realistic covert communication
nsF5, p=3, d=0.55	medium	Moderate strength
nsF5, p=2, d=0.35	weak-medium	Lower p is less efficient, leaves more traces
nsF5, p=2, d=0.85	strong	Heavy embedding, obvious footprint
matrix, p=3, d=0.50	medium	Compare another method
matrix, p=2, d=0.80	strong	High-density plain matrix embedding

v1.4 adds +6	-	nsF5 p3 d0.40; matrix p3 d0.40/0.60; lsb d0.30/0.50/0.70

All images resize to 512x512 grayscale; features come from C++ and hiding variants are produced by the C++ embedding path, so training and inference stay consistent.

Tip

The "p, density" on the right is exactly the "embedding strength" from 8.1: **the stronger the band, the more is hidden, the clearer the statistical trace, the easier the classifier.** That is also why weak densities are harder to detect (Fig. 8-5).

8.4 How to Read the Training Script: train_model.py

`python src\train_model.py` is one honest evaluation pipeline:

1. Load the table and drop rows with missing features;
2. Reserve 25% as the final test set by `photo_id` (those photos never enter training);
3. On the training pool, run 5-fold cross-validation comparing **logistic regression / random forest / gradient boosting / XGBoost**;
4. Pick the best CV model, carve 20% of the training pool as a calibration set, and choose a threshold;
5. Retrain on the full training pool and report AUC / accuracy on the held-out test set;
6. Report per-(method, p, density) detection rates;
7. Save the model and thresholds to `models/stego_classifier.joblib`.

8.4.1 Four Classifiers, Each an Approach (ideas + real comparison)

Step 3 compares four models - really just three ideas:

- **Logistic regression (LR)**: the "draw a line + sigmoid to probability" from Chapter 7. **Most intuitive, most stable, most explainable**; con: the line is straight, it cannot bend.
- **Random forest (RF)**: **a crowd of trees voting**. Each tree asks a chain of "if this feature > some value, go left, else right." Many trees trained on different samples and features, then majority vote. **Stable, resists overfitting**.
- **Gradient boosting / XGBoost (GB/XGB)**: **relay to fix errors**. Not many trees at once - one after another, each tree fixes what the previous got wrong. XGBoost is the "upgraded" version with extra anti-overfitting rules: fast, often best.

```
def lr(seed=0):  return make_pipeline(StandardScaler(), LogisticRegression(
    max_iter=2000, C=0.1, random_state=seed))
def rf(seed=0):  return RandomForestClassifier(n_estimators=500, max_depth=None,
    min_samples_split=2, n_jobs=-1, random_state=seed)
def gb(seed=0):  return GradientBoostingClassifier(n_estimators=300,
    learning_rate=0.05, max_depth=3, random_state=seed)
def xg(seed=0):  return XGBClassifier(n_estimators=400, learning_rate=0.05,
    max_depth=4,
                    subsample=0.9, colsample_bytree=0.8, eval_metric="logloss",
                       random_state=seed, n_jobs=-1)
```

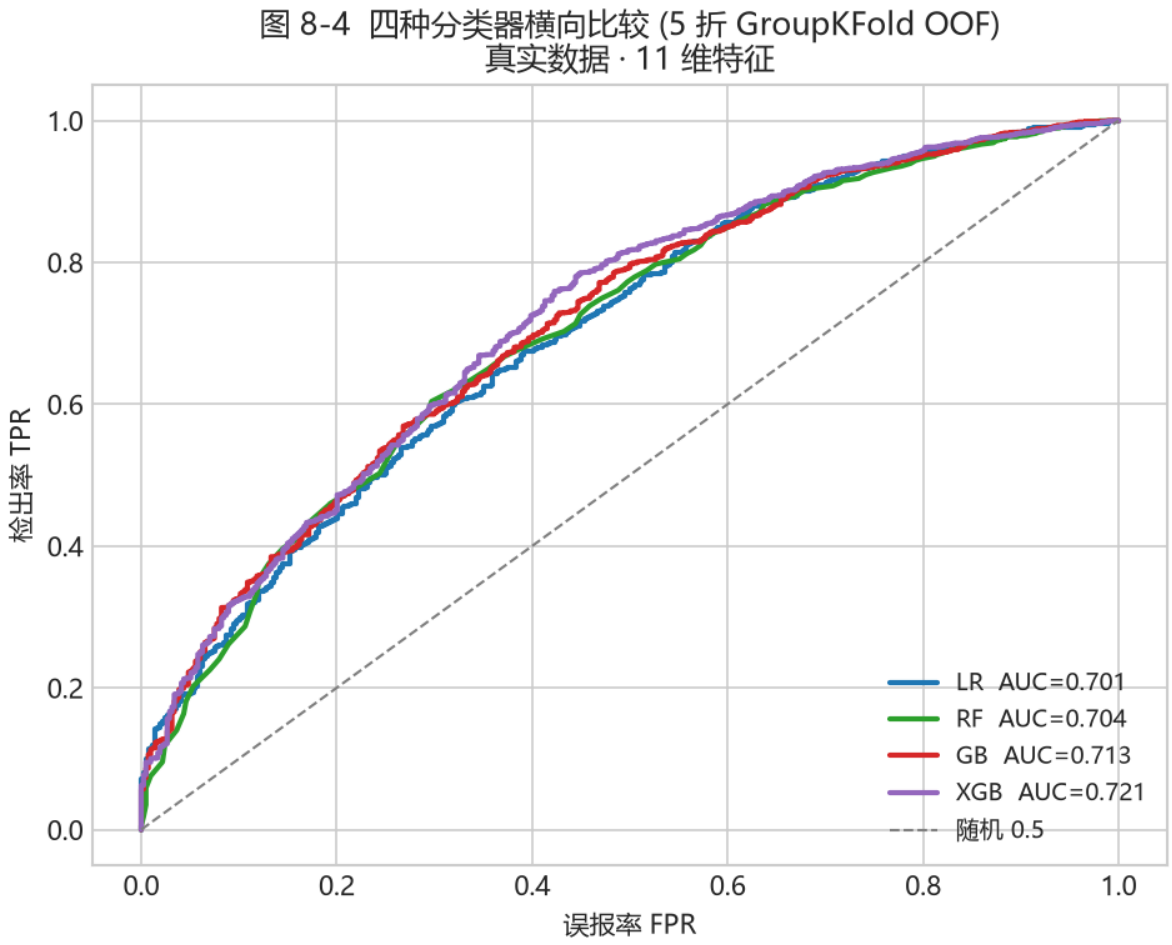

图 26: Fig. 8-4 classifier ROC comparison

Fig. 8-4 (real data 11-D, 5-fold GroupKFold OOF: the four classifiers' ROC curves nearly overlap, all AUC between 0.70-0.72. This means ***on these 11-D features the method matters little - the bottleneck is more in the features than the classifier****, which backs up 8.1: get the features right first, then pick a classifier)*

If you want to go deeper

A tree splits by picking one feature and one threshold that splits samples into two piles, each as "pure" as possible. Purity uses **Gini impurity** $G = 1 - \sum_c p_c^2$ or information entropy. `n_estimators=500` means plant 500 trees; `learning_rate=0.05` is "step gently". No need to memorize - just know "trees split toward purity" and "XGBoost is more restrained."

8.4.2 Which Feature Matters More: Look at the LR Coefficients

Since the classifiers differ little, the **features** are what matter. The weights w learned by logistic regression directly tell us how much each feature contributes and in which direction:

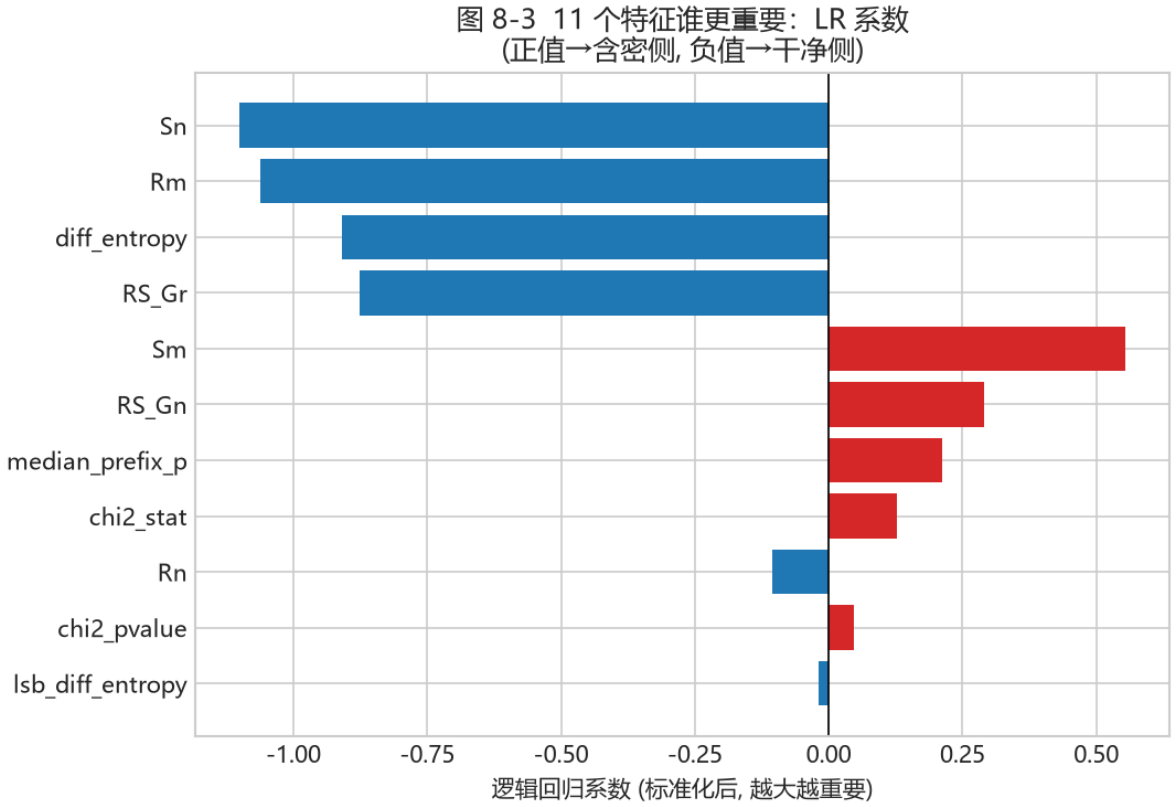

图 27: Fig. 8-3 feature importance

Fig. 8-3 (real data 11-D, standardized LR coefficients: positive (red) pushes toward "hiding", negative (blue) pulls toward "clean"; larger absolute value = more important. Sn, Rm, diff_entropy, RS_Gr contribute a lot (negative); Sm, RS_Gn, median_prefix_p, chi2_stat contribute a lot (positive))

Reading the figure:

- **Large absolute** coefficients = big influence on the judgement;
- **Positive** coefficients = larger feature value means more hiding; **negative** the opposite;

- This echoes Fig. 8-1: a large Gn means more clean (hence the coefficient direction).

8.4.3 A Crucial Detail: OOF and "Do Not Test Yourself"

The cross-validation in step 3 is not only to compare models - it also produces a clean, trustworthy score. Its trick: **each sample is predicted only by a fold that never saw it**, and those predictions are collected (called OOF, out-of-fold). This score is honest - not contaminated by "training and testing on yourself."

Watch out

The calibration set is for choosing a threshold; the test set is used exactly once. Re-tuning the threshold on the test set dirties it. The project's **train/calibration/test** three-layer structure is the rigorous practice industry uses.

8.5 Reading Results: ROC, AUC, and Per-Density Rates

Beyond the overall AUC (Fig. 7-6), look at **per-band (method, p, density) detection rates**, because the bands differ a lot.

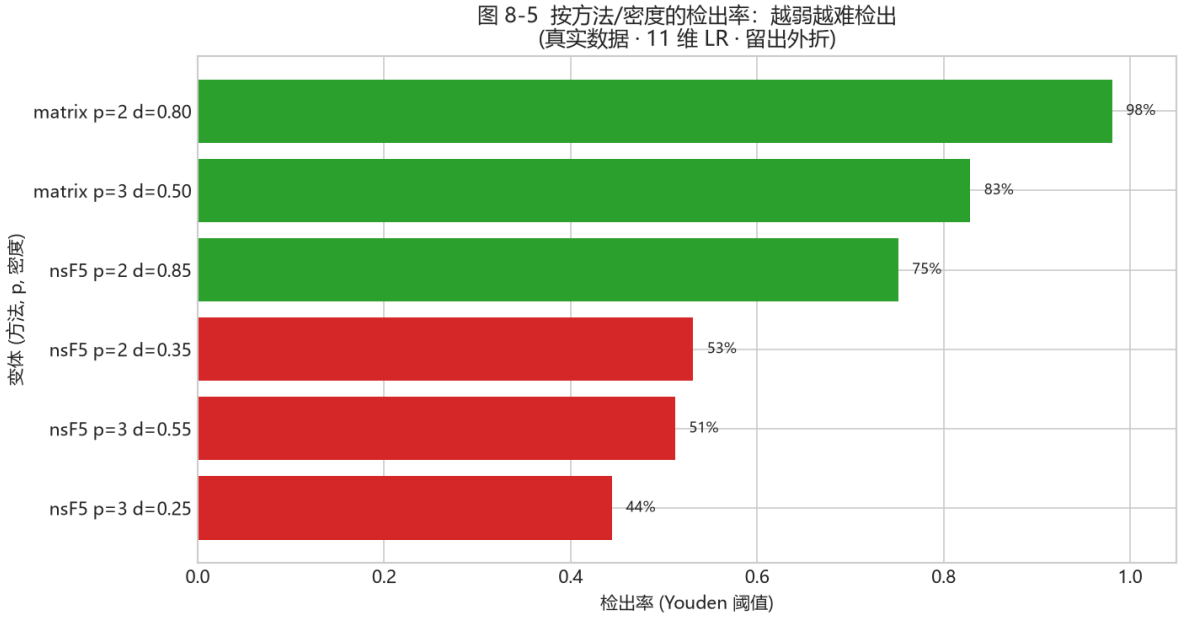

图 28: Fig. 8-5 per-density detection

Fig. 8-5 (real data 11-D LR, held-out folds, Youden threshold: detection rate grouped by (method, p, density). ***Strong densities (matrix p2/p3) near 90%+, weak densities (nsF5 p3 d0.25) clearly lower (around 50%+)*** *- direct evidence that "the weaker the embedding, the harder to detect")*

Back to the code

Compare `thesis/data/det_density.csv`: weak nsF5 p3 band around 48-64%, strong matrix p2/p3 75-99%. This confirms 8.3 - **small density means a weak statistical trace, a weak feature signal, so harder to detect** - not mysticism.

8.6 Sensitivity: Switching "Strict/Loose" in the GUI

`src/ml_predict.py`'s `MLPredictor` preprocesses the image (grayscale -> 512x512), then auto-selects 11-D or 143-D by the model's stored feature count, and outputs a hiding probability. "Sensitivity" just means **switching the decision threshold**:

Sensitivity	ML threshold	Effect
Strict (low FP)	threshold_low_fp	Fewest clean-image errors; more weak hidden missed
Balanced	Youden threshold	Balanced detection vs false positives; default
Loose (high detection)	Youden x 0.75	Catches more weak densities; false positives rise

Against the code - `ml_predict.py::_threshold`.

```
if self.sensitivity.startswith("严格"): return float(d.get("threshold_low_fp", d[
    "threshold"]))
if self.sensitivity.startswith("宽松"): return max(0.05, float(d["threshold"]) *
    0.75)
return float(d["threshold"])
```

Tip

"Loose" does not use a new model - it just **lowers the decision threshold by 25%**. A lower threshold is more aggressive: catches more weakly-hidden images, but raises false positives - the "threshold = strict or loose" from Fig. 7-7/7-8, made concrete.

The GUI's "Analyze" page shows the heuristic probability and the ML probability **side by side** and asks you to cross-check, not trust one number - a deliberate honesty feature.

8.7 Experiments: From Training to Single-Image Judgment

The full pipeline (minutes to tens of minutes, machine dependent)

```
# 1) generate a dataset (point it at your own photo folder)
python src\make_dataset.py

# 2) train and evaluate
python src\train_model.py

# 3) single-image ML judgment
import sys; sys.path.insert(0, "src")
import numpy as np
from PIL import Image
from ns5_core import embed_string
from ml_predict import get_predictor
a = np.asarray(Image.open("img/cover.png").convert("L"))
s, _, _ = embed_string(a, "ML␣test", method="nsF5", p=3)
print(get_predictor().predict(s))
```

Try it

Run an honesty experiment: judge a clean photo (should be near clean), then hide a weak-band message (nsF5 p=3, density near 0.25) and judge again. Watch whether the probability **rises only slightly** - that is the README warning about weak densities, live.

Against Chapter 7 formulas

Step 3's `predict()` is internally: feature x -> weighted sum $z = w \cdot x + b$ -> sigmoid to probability -> compare with threshold for the verdict.

8.8 v1.4.0 Update: SRM, 143-D Features, and Dual Models (2026-09)

Sections 8.1-8.7 describe the v1.0-v1.3 route: 11-D features with LR/XGB, AUC 0.75-0.79. v1.4.0 upgraded along three lines: SRM high-pass preprocessing, 143-D v2 features with 12 variants, and finally the dual LightGBM model strategy. The story follows the actual evolution, so the README's experiment tables make sense.

8.8.1 SRM High-Pass Filtering: Same-Source Gain, Cross-Source Loss

SRM (Spatial Rich Model) is a set of **high-pass filter kernels**. `src/srm_filter.py` implements 30 standard kernels, filters the image, takes the maximum absolute response per pixel, clips to +/- 4, and rescales into an "enhanced image" that feeds the existing feature extractors. CPU (numpy)

and GPU (torch) implementations; switches are `make_dataset.py --preprocess srm` and the GPU `use_srm=True`.

Metric	Same-source baseline	Same-source + SRM
5-fold CV-AUC	0.7594	0.7922
Held-out test AUC	0.7811	0.8085
Weak nsF5 p3 d0.25 detection	51.9%	62.5%
nsF5 p2 d0.35 detection	76.0%	90.4%
Low-FP stego detection	41.2%	50.3%
Clean false positives (low-FP thr)	9.6%	9.6%

Watch out

The gain only appears within **same-source** data. In cross-source (campus + full BOSSbase), SRM pulls CPU test AUC from 0.741 down to 0.704 and GPU validation AUC from 0.651 down to 0.572 - high-pass filtering also flattens the differences between cameras/compression sources. So the default model is trained on merged data without SRM, while the SRM campus model is kept separately as campus_srm.

8.8.2 The v2 143-D Feature Set and 12 Variants

`src/featurize_v2.py` adds: mean / absolute mean / standard deviation over 30 SRM residuals (90-D), 20 full-image prefix chi-square p-values, 20 LSB-prefix p-values, plus texture_noise and est_rate (2-D): 11 + 90 + 20 + 2 + 20 = 143. Variants grow from 6 to 12: add nsF5 p3 d0.40, matrix p3 d0.40/0.60, and lsb d0.30/0.50/0.70.

Strict A/B (same test-photo groups, 5-fold GroupKFold OOF): the v2 dataset alone adds +0.019; adding 143-D adds another +0.027 (OOF 0.8143, held-out 0.8278), about +0.05 AUC total.

Tip

143-D is not extra from nowhere - it measures 8.2's three groups of rulers **finer and from more angles**: BASE 11 is the backbone; SRM 90 are residual statistics; PREFIX 20 + LSB-PREFIX 20 are the chi-square test "split into 20 segments + LSB version". Still measuring the same thing - the statistical trace.

8.8.3 Real JPEG Clean Images Expose a Deployment Problem - and the Fix

The v2 logistic model scored well inside its training distribution, but on real JPEG clean photos it **flagged almost everything as 1.0**: SRM residuals react strongly to JPEG high-frequency noise. The fix: add 414 real JPEG clean photos as independent `photo_id` (fully disjoint from the original training groups), producing `dataset_campus_v2_jpeg.csv`, then grid-tune LightGBM. The tuned LGB became default: held-out AUC 0.8946, 8-split average 0.9085, weak nsF5 p3 d0.25 detection up from 67.9% to 85.3%.

Tip

This is a live **distribution shift** case: the model learns well on "campus PGM/BMP grayscale" then breaks on "real JPEG camera noise." The project fixes it by adding real JPEG clean images to training - **training data must cover the real deployment scenario**, or the score is self-congratulation.

8.8.4 Dual-version Deployment: 143d Robust vs 53d Interpretable

Interpretability experiments showed 73% of the 143-D gain comes from 31 explainable features (BASE 11 + PREFIX 20); the SRM 90 dimensions average single-feature AUC 0.50-0.52 (near random) yet serve as a buffer that filters JPEG noise and improves OOD robustness. So two models are kept:

Dimension	143d default (robust)	53d interpretable
Features	BASE11 + SRM90 + PREFIX20 + LSB-PREFIX20 + TEX/EST2	Same minus SRM90 (53-D)
8-split mean AUC	0.9085	0.9227
Held-out AUC	0.8946	0.9100
Weak nsF5 p3 d0.25	85.3%	87.2%
Real-JPEG clean OOD	1/8 fp (median 0.011)	3/8 fp (median 0.028)
Best for	Deployment / mixed domains / real images	Papers / defenses / teaching / per-image explanations

`MLPredictor()` loads 143d by default; pass `model_path` to switch to 53d. Inference auto-selects 11-D or 143-D from the model metadata. GUI model switching is still planned; for now, switch via the API:

Switching between the dual models

```
from ml_predict import MLPredictor

# default: 143d robust model (models/stego_classifier.joblib)
pred_143 = MLPredictor()

# 53d interpretable model (SRM 90 removed; higher AUC)
pred_53 = MLPredictor(
    model_path='models/stego_classifier_v2_jpeg_lgb_51d.joblib',
    clip_outliers=False)

r = pred_53.predict(img)
# {'probability': ..., 'verdict': ..., 'threshold': ...}
```

Try it

Experiment: train on dataset_campus_v2_jpeg.csv (set $env:DS_FILES), then compare 143d vs 53d on real JPEG clean photos and on weak nsF5 p3 d0.25 images. Put the AUC and OOD behavior into your report.

Watch out

The v2/53d models are only reliable near their training distribution. For a brand-new camera or compression source, run a small OOD test before deployment.

8.9 Summary and Self-Check

One sentence: under "small samples + weak signals", **domain features + simple classifiers** are more reliable than a raw CNN; and the **features set the ceiling** - the classifier merely mines the signal in them.

- The 11 features = RS family (Gn collapse) + chi-square family (gray pairs flattened) + entropy/randomness baseline (spotting naturally messy images); see Fig. 8-1/8-2.
- The four classifiers (LR/RF/GB/XGB) differ little on the 11-D features (Fig. 8-4); what really separates is **features** (Fig. 8-3) and **data**.
- **Train/calibration/test three-layer structure + GroupKFold** is the skeleton of experimental trust;
- The weaker, the harder to detect (Fig. 8-5); detection has a physical ceiling;
- Cross-check ML probability with heuristics; since v1.4 dual models are strong, but real-JPEG OOD validation remains mandatory (8.8).

Think about it

One photo has 7 samples. Why is a random 80/20 split not acceptable? Design a minimal example showing how much leakage can inflate AUC. One layer deeper: if you lower the "Loose" threshold again in 8.6, how does the false-positive rate change? Why are "catch more" and "false-positive less" always in tension? Finally: if you had 100,000 independent photos, would you still stick with "features instead of a raw CNN"? Why?

Chapter 9 - Engineering: C++, GPU, GUI, and Tests (Week 10)

Try it

Goals: see how algorithms become software; understand why C++ and GPU acceleration exist and how cross-language consistency is guaranteed; run the tests and trace the CI.

9.1 Architecture: Layered, Not Coupled

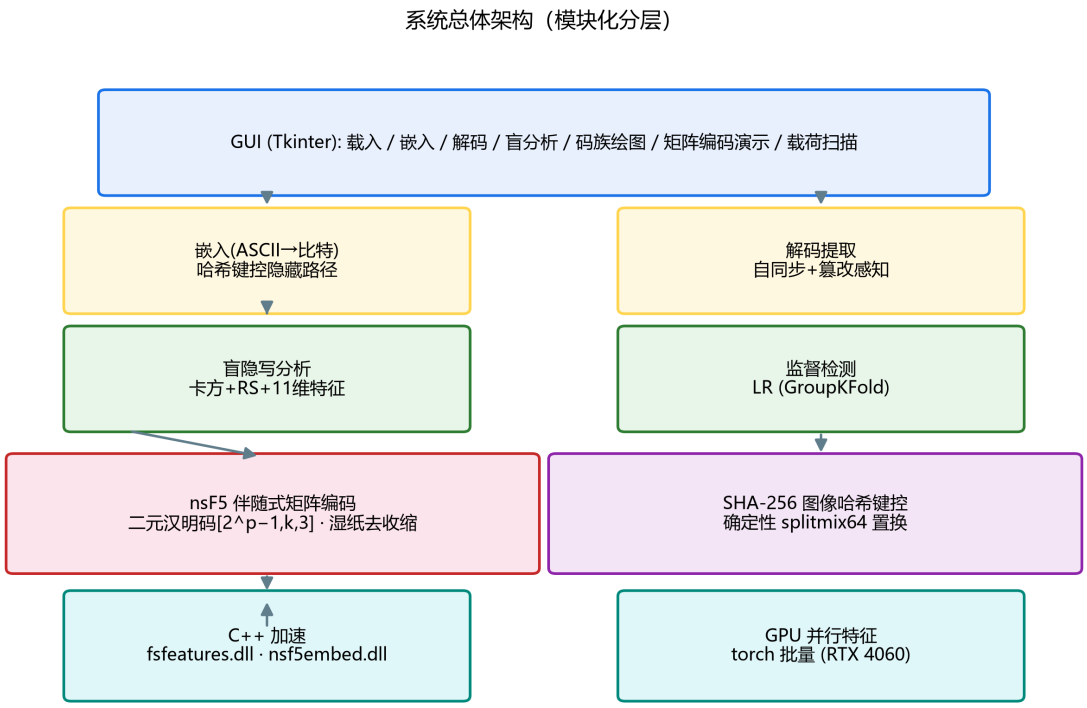

图 29: fig-5

Fig. 9-1 System architecture: acceleration layer -> algorithm/security core -> features -> GUI (thesis figure)

- **Acceleration layer:** cpp/fsfeatures.dll (features), cpp/nsf5embed.dll (embedding/shuffling);

- **Algorithm core:** ns5_core.py - Hamming codes, wet paper, hash keying, embed/extract;
- **Feature layer:** steganalysis.py, efficiency.py, make_dataset.py, train_model.py, ml_predict.py, featurize_v2.py, srm_filter.py;
- **UI layer:** gui.py + matrix_demo.py + scan_panel.py - one-click access to everything.

Layering matters because the algorithm layer does not depend on the GUI (it runs on servers and in CI), and the acceleration layer falls back to pure Python when a DLL is missing, so the features always work.

9.2 C++ Acceleration: Hot Paths and How We Know They Are Right

Python is slow at elementwise loops, and this project has two classic hot paths: **deterministic permutation** (shuffling up to 16 million positions) and **feature extraction** (RS/chi-square/entropy statistics per image). Moving them into C++ DLLs gives orders-of-magnitude gains: permutation speedup up to 263x (40962 image: 12.3 s in Python -> 223 ms in C++).

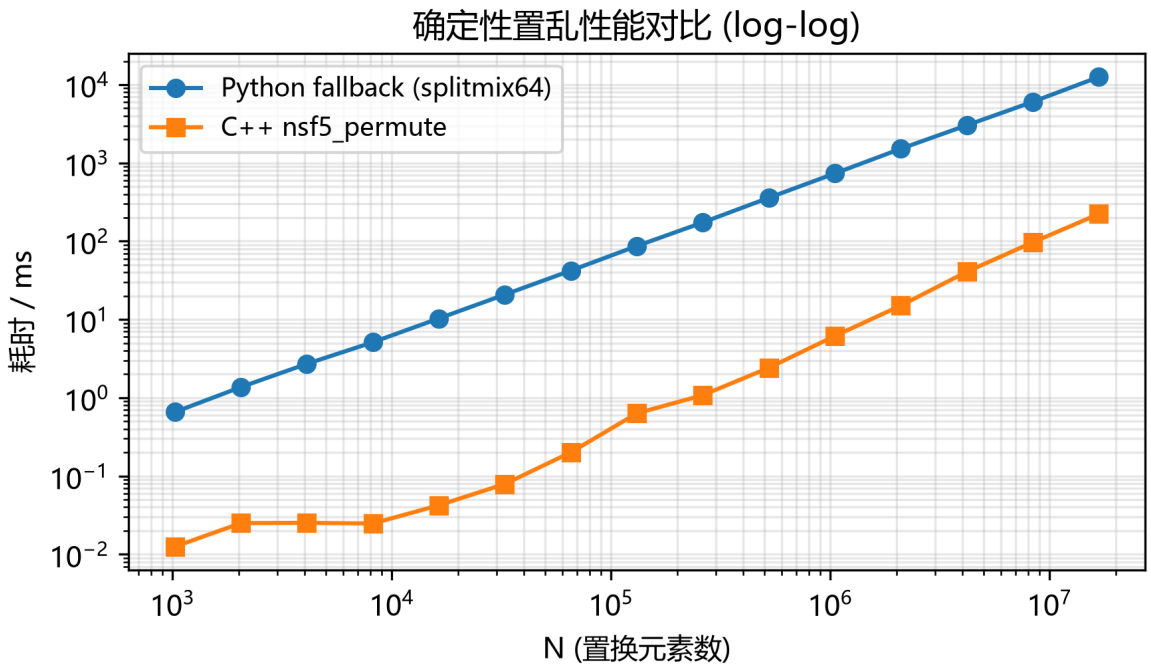

图 30: fig-6

Fig. 9-2 Deterministic permutation: Python vs C++ (log-log axes, thesis figure)

Speed is worthless unless the result is identical. Python calls the DLLs through ctypes, and the project guards correctness with self-checks:

- `cppembed.py` embeds the same image through C++ and Python paths and compares outputs pixel by pixel;
- `fsfeatures.py` check_against_python() compares every C++ feature with the Python reference;
- When the DLL is missing or fails to load, the code falls back to the same Python algorithm, keeping embed/decode reversible everywhere.

Watch out

Cross-language consistency is the prerequisite for acceleration and a classic debugging minefield: bitwise operations, rounding, and overflow differ between C++ and NumPy. If you ever see "embeds with DLL, cannot decode without it," run `cppembed.selfcheck()` and `fsfeatures.check_against_python()` before touching the algorithm.

9.3 GPU Version: Batch-Vectorizing Features

The `gpu/` directory rewrites feature computation as PyTorch tensor operators: load a batch of 512x512 grayscale images, compute histograms, RS, chi-square, and entropies in parallel on the GPU, then return results to the CPU. v1.4 adds `gpu/featurize_v2_gpu.py` for the 143-D v2 features (including SRM convolutions). The v1 pipeline extracts 2,070 images in about 5 seconds (410 images/s) and matches the CPU reference bit-for-bit (float error 1e-7).

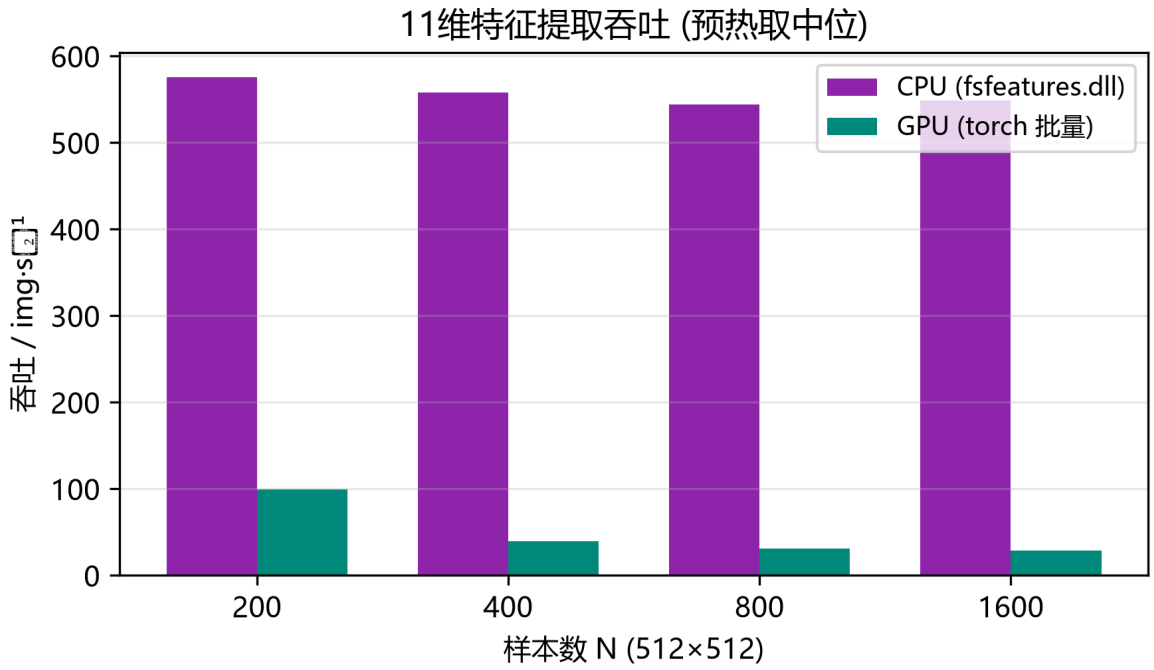

图 31: fig-7

Fig. 9-3 v1 11-D feature throughput: CPU (C++) vs GPU (torch batch) (thesis figure)

Read the code

Read the self-check in `gpu/featurize_gpu.py`: it compares GPU results element by element with `src/fsfeatures.py`. Bit-level consistency is what makes the dual implementation safe. Then read the README discussion of throughput limits: for small batches, host-device transfer can make GPU slower than C++ - an honest measurement, not a marketing claim.

9.4 GUI: Turning a Research Tool into a Teaching Application

`src/gui.py` is organized around Embed -> Decode -> Analyze. Two teaching panels stand out:

- **Matrix coding demo** (matrix_demo.py): click a block, watch s, m, and d update live, see the matched H column highlighted, and verify H*x' = m after the flip;
- **Payload scan** (scan_panel.py): drag payload from 0 to 0.4 and watch chi-square p, RS estimate, and ML probability curves refresh as the image is re-embedded at each density.

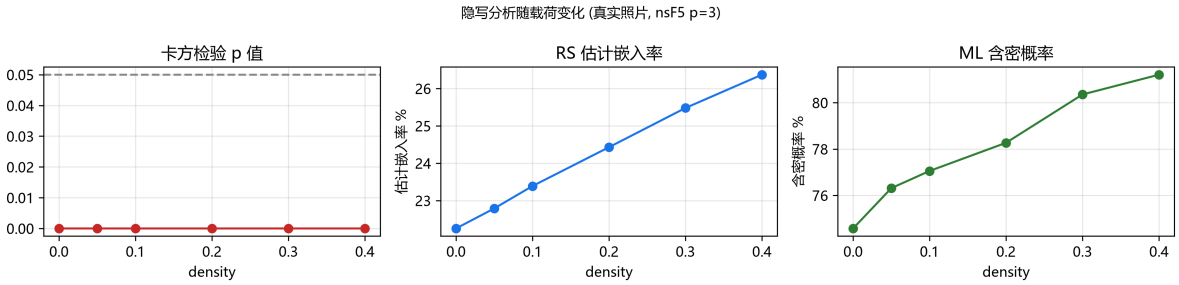

图 32: fig-8

Fig. 9-4 Payload scan: detectability grows with embedding density (thesis figure)

Watch out

Note the wording in the GUI: a single image has no true "AUC" (AUC needs a set of positive and negative samples), so the scan panel plots model probability as a trend illustration. Distinguish "single-image probability" from "dataset metrics" whenever you read software output.

9.5 Tests and CI: Refactor Without Regret

Test file	Verifies	Mental model

test_core.py	Hamming matrices, embed/decode round trips, wet paper, wrong passwords	Algorithms still work
test_steg.py	Blind analysis separates clean from stego	Analysis still works
test_false_positive.py	Clean images are not flagged (regression)	False positives stay controlled
test_gui.py	Window builds, loads images, previews	UI still works

`.github/workflows/ci.yml` runs core tests and builds wheel + sdist on every push/PR to main; pushing a `v*` tag creates a GitHub Release. Tests + packaging + auto-release is the standard way an algorithm project becomes a reusable tool.

9.6 A Code-Reading Route (Use as Needed)

1. Entry point: src/run_e2e.py - the shortest path through every module;
1. Data layer: image_io.py - how arrays enter and leave;
1. Embedding layer: read ns5_core.py top to bottom (hash, shuffle, Hamming, wet paper, high-level API);
1. Analysis layer: trace analyze() in steganalysis.py backwards through each statistic;
1. ML layer: train_model.py main flow -> featurize_v2.py / srm_filter.py (v1.4) -> ml_predict.py dual-model inference;
1. Acceleration layer: cppembed.py / fsfeatures.py ctypes bindings and self-checks;
1. UI layer: find a button callback in gui.py and follow it to the algorithm function.

Think about it

Why does the comment in ns5_core.py insist that changing the permutation algorithm breaks reversibility? Combine with the v1.2.1 fix (automatic Python fallback): what catastrophe happens if Python and C++ permutations differ?

9.7 v1.4.0 Update: SRM, Multi-Source Data, and Model Engineering (2026-09)

Several engineering modules changed in v1.4.0. Learn the new directory first:

New file / module	Role	What to study
src/srm_filter.py	30 standard SRM high-pass kernels; numpy/torch dual implementations	Kernel list, normalization, enhanced-image pipeline
src/featurize_v2.py	143-D v2 feature assembly on CPU	ALL_FEATURE_NAMES, featurize_v2()
gpu/featurize_v2_gpu.py	GPU batch 143-D features and consistency self-check	extract_features_v2_gpu()
src/make_dataset.py extensions	Multiprocessing -j, SRM/feature-set/variants options	Per-image parallelism, identical row output
src/train_model.py extensions	DS_FILES multi-dataset, stacking, LGB tuning	Dual-model saving and thresholds
gpu/make_imageset.py extensions	memmap disk write, –out/–id-offset	Multi-source partitioning without OOM

On the data side, v1.4 established a pure campus-photo baseline `data/campus_jpg` (414 photos; earlier DIP4E textbook images were removed) and added the standard steganalysis benchmark BOSSbase 1.01 (10,000 512x512 grayscale PGM images) for multi-source training. CPU merged training used 72,898 samples (held-out AUC 0.741); GPU merged training used 52,070 samples (validation AUC 0.712); BOSSbase alone reached only 0.644 on GPU - a harder benchmark with weaker signals. `make_dataset.py` now parallelizes across images (5.6x speedup on 16 cores), and GPU image sets use memmap writes to avoid OOM.

Tooling changed too: the license moved from MIT to Apache-2.0 with a NOTICE file and third-party attributions, and README/pyproject now declare v1.4.0. The pixel-level and feature-level self-checks between C++ and Python remain in place.

Try it

Run `python gpu\featurize_v2_gpu.py` self-check. Then follow the README to generate one source from `data\campus_jpg` and one from `data\BOSSbase_1.01`, and compare three models: campus only, BOSSbase only, and merged. That is the most instructive data-domain experiment in v1.4.

Chapter 10 - Capstone: From Reading to Building (Weeks 11-12)

Try it

Goals: complete one demonstrable improvement experiment in two weeks. Three directions are given below, easy to hard; you may combine them. **But before starting, remember the skeleton of "how to make an experiment count" (Fig. 10-1).**

10.0 Experiment Methodology: First Learn "How to Count as Valid"

Whether an experimental conclusion is trustworthy does not depend on how hard you work, but on whether you keep **evaluation honesty**. This is the skeleton Chapters 7-8 stress, and for capstone it decides every conclusion you draw:

Fig. 10-1 (`train_model.py`'s real steps - also the skeleton for your capstone: data -> reserve test set by photo_id -> 5-fold GroupKFold OOF on the training pool -> pick the model exactly once -> calibration set picks thresholds -> retrain on full -> held-out test evaluated once -> save)

Reading the figure

- **The test set is touched exactly once**: re-tuning the threshold on it dirties it; the score is no longer trustworthy; - **Calibration set**: carved from the training pool purely to pick thresholds (Youden / low-FP); - **OOF (out-of-fold)**: each sample is scored only by a fold that never saw it; used to pick the model - preventing "proving yourself with your own training". Whichever direction you choose below, **come back to this figure**: first reproduce the baseline (full Fig. 10-1 flow), then make a small change, and give an honest comparison with `GroupKFold`. Remember the line: **"changed A, result got better" is not enough; prove it truly got better using the train/calibration/test three-layer structure + grouped split.**

10.1 Direction A: Reproduce and Critically Verify (most stable)

Pick 1-2 conclusions from the paper/README, rerun them yourself and try to "refute" them:

- Reproduce the code-family figure: does theory $\alpha(p)$ match measurement? Where does deviation come from at large p?

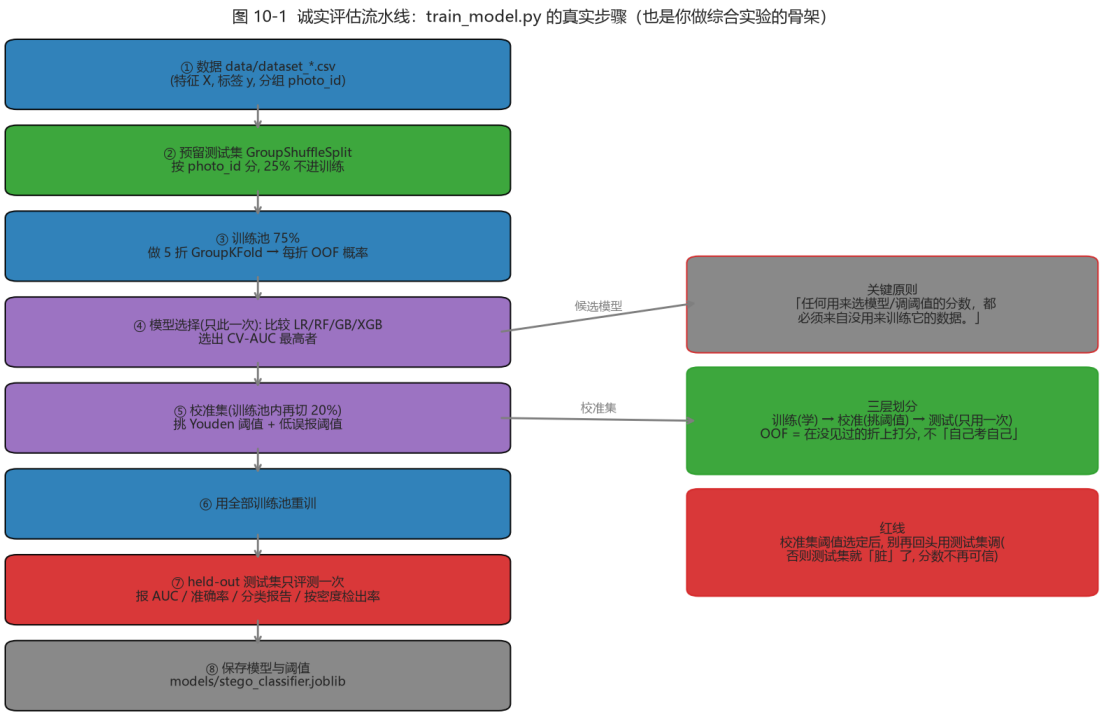

图 33: Fig. 10-1 honest evaluation pipeline

- Reproduce a detection experiment: train on your own photos, compare the v1 11-D baseline (about 0.75-0.79) with the README v1.4 dual models, and observe how photo style affects AUC;
- Reproduce the tampering experiment: change 1 pixel, 1 row, 1 block - what are the decode failure rates?

Watch out

Reproducing is not copying. Its value is **understanding where each number comes from**: is AUC OOF or held-out? Was `photo_id` grouping used? How was the threshold set? These (Chapter 7) decide whether you can ”critically verify” rather than ”blindly trust the paper”.

Acceptance	**Pass criterion**
Experiment log	record environment, commands, parameters, raw output
Figures	at least 2 self-drawn curves/comparisons with labeled axes and legends
Conclusion	distinguish ”verified the paper” vs ”inconsistent with the paper” and give reasons

Reproducible	one-click script or step-by-step commands that others can rerun

10.2 Direction B: Feature Extension (most satisfying)

- **Support Chinese/UTF-8 messages**: change `encode_string()`'s ASCII encoding to UTF-8 (note the length header is now bytes not chars);
- **Color channel extension**: currently only the R channel is used; study how to allocate capacity/security across R/G/B;
- **New feature**: add one statistic you think separates clean/stego on top of the 11-D or 143-D set, and evaluate whether AUC truly rises with GroupKFold;
- **Compare a new algorithm**: implement naive LSB replacement and compare Gn collapse and detection rate against nsF5 at equal payload.

Watch out

After every extension, run `test_core.py` and `test_steg.py` to keep the "embed -> decode" roundtrip valid. When you change the encoding, the decoder must change in lockstep - the most common beginner mistake.

Tip

"New feature" extensions are the easiest to feel good about yourself: add a feature, AUC ticked up, done. Please go back to Fig. 10-1 - do the A/B on the **same test-photo groups and 5-fold GroupKFold OOF**, or your "+0.01" might just be random noise (the practical lesson of 7.8 leakage/generalization).

10.3 Direction C: Teaching-Demo Repackaging

Turn the GUI's matrix-coding demo into a set of explainable textbook pages/animation scripts, e.g.:

- Step-by-step animation: random block -> show H -> compute s -> give m -> get d -> highlight -> verify;
- Quiz mode: random questions asking you to hand-compute "which bit to flip", auto-graded;
- Compare mode: the same message embedded with LSB, matrix p3, nsF5 p3, showing changed count and Gn side by side.

10.4 Paper-to-Code Reading Table

Paper section	Corresponding code	Master before reading
Ch. 2 Foundations	ns5_core.py / steganalysis.py	LSB, Hamming, wet paper, chi-square/RS
Ch. 3 Requirements	README overview	the stego-detection game view
Ch. 4 Design	arch figure + ns5_core.py	layered architecture, hash keying flow
Ch. 4.5 Acceleration	cppembed.py / fsfeatures.py	ctypes and consistency self-check
Ch. 4.6 GPU	gpu/*.py	tensorization, batching, throughput limits
Ch. 5 Experiments	run_e2e / make_dataset / train_model	GroupKFold, AUC, Youden

Tip: the paper draft has been updated to thesis/thesis1.pdf (v1.4.0 experiments, incl. SRM/143d/53d and multi-source data); the table above uses the early paper's sections as an example - cross-reference the new paper's table of contents.

10.5 Common Presentation/Defense Questions & Answers

Frequent question	Answer points
Why can LSB hiding be found?	Adjacent gray pairs flattened (chi-square) + LSB structure collapse (RS); intuition first, statistics second
Why does matrix coding "change less"?	n=2 -1 positions carry p bits; distinct H columns -> any syndrome difference maps to one position
What is better about nsF5 vs F5?	Wet-paper pre-marks wet points; solve only on dry points; no shrinkage, no re-embed
How does wet paper solve it?	GF(2) linear equation H_dry·y=d; try single/double column, then Gaussian fallback
Why not a raw CNN?	small samples, weak signal, no generalizable features; features + simple model more stable (AUC evidence)
What does AUC 0.756 mean?	ranking okay but weak density hard; AUC vs "detection at a specific threshold" are different things
What does hash keying prevent?	predictable/copyable positions; senses ordinary tampering, but is not a keyed MAC
Risk of C++/GPU speedup?	yes: must do cross-language consistency checks, else embed/decode becomes irreversible

How do you prove ”your model is better”?	back to Fig. 10-1: same test photos, 5-fold GroupKFold OOF, train/cal/test separation; single-image probability != dataset AUC

10.6 Delivery Checklist (final week)

- Improved code diff (be able to explain every change);
- Experiment script + raw output (unprocessed, not cherry-picked);
- 2-3 figures (efficiency / ROC / density detection / tampering, any);
- 3-5 page report: problem -> method -> result -> limitation -> next step;
- 5-minute demo: a one-run flow from embedding to analysis;
- Answer any three questions from 10.5 without notes.

Think about it

For your improvement experiment, which ”honesty guarantee” of Fig. 10-1 will you use to make ”the result is credible”? If someone challenges ”did you tune the test set repeatedly to get this?”, how do you respond? (Hint: state the ”test set used once” red line and show the OOF/calibration results.)

Chapter 11 - Boundaries, Ethics, and What Comes Next (Read Along the Way)

11.1 Legal and Ethical Boundaries

Steganography is double-edged: it supports watermarking, media provenance, and covert authentication, but it can also be abused for malicious covert communication. The right posture for learning and research is:

- Experiment only on your own data, public datasets, or authorized material;
- Use the tooling for detection and forensics rather than concealment;
- Report limitations and false-positive risk honestly; never overstate detection power;
- Do not use other people's private photos as practice carriers; de-identify and authorize data.

Watch out

The steganalysis half of this project (chi-square/RS/ML) is the shield: monitoring and forensics need it to find covert channels. Think about your data and purpose before studying attack surfaces.

11.2 Known Limitations (Understanding Boundaries Is Understanding the Project)

- Messages are ASCII-only by default; Chinese/UTF-8 requires an extension;
- Color images use only the R channel, underutilizing capacity;
- Keying is not a keyed MAC, so a strong attacker who re-embeds can update the header hash;
- The v1 dataset still starts from a limited set of independent photos (414 campus, later expanded with BOSSbase); weak-density and cross-source detection remain hard;
- The v2/143-D model was over-aggressive on out-of-distribution real JPEG clean images; the fix requires JPEG clean samples in training - always run OOD tests before deployment;

- SRM helps only within the same source domain and hurts across heterogeneous sources;
- The dual models trade off robustness and interpretability (143d: 1/8 OOD FP; 53d: higher AUC but 3/8 OOD FP); GUI switching is planned, use the API today;
- Heuristic analysis outputs a ”stego tendency probability,” not a strict statistical probability;
- Pixel-domain nsF5 is incompatible with lossy formats; JPEG-domain work needs the yccstego-style DCT coefficient design.

11.3 A Map of Modern Information Hiding and Steganalysis

Direction	Representative ideas / methods	Distance from an undergrad
Content-adaptive embedding	HUGO / WOW / UNIWARD: hide where texture is complex	Requires optimization background
Deep-learning steganography	Encoder-decoder models trained end-to-end	Reproducible entry projects exist
Deep-learning steganalysis	SRNet / large-scale CNNs	Needs big data and compute
JPEG-domain hiding	yccstego: nsF5 on quantized DCT coefficients of Y	Direct extension of this project
Robust/reversible watermarking	Trade invisibility, robustness, capacity	Engineering-heavy; industry-relevant

If your interest is sparked, the next steps are concrete: read the original wet-paper paper (Fridrich et al.), study yccstego’s quantized-DCT pipeline, then pick one modern algorithm for a literature review - Appendix E gives entry points.

11.4 Closing Words

Twelve weeks ago steganography may have sounded like ”hiding words in images.” Now you can explain why it depends on carrier redundancy, why direct LSB changes leave statistical fingerprints, how Hamming codes carry the most information per modification, how wet paper coding removes shrinkage, and how machine learning states its confidence honestly. Those ideas connect one project in F:\Steganography to a way of thinking shared by information security, probability, and machine learning.

Try it

Final exercise: without notes, write 500 words explaining to a classmate why nsF5 is better than naive LSB. Then open the README and compare. If you can write it, you have finished.

Appendix A - Glossary

Arranged roughly in reading order. Mastering the bold-faced terms covers almost everything in this handbook.

Term	**Also known as**	**One-line meaning**
Steganography	-	Hiding a secret inside an innocent carrier so the act of communication is hidden
Steganalysis	-	Deciding whether a carrier hides a secret, from statistics or learning
cover / stego	carrier / embedded image	Original image / image after embedding
LSB	least significant bit	Lowest bit of a pixel; flipping it is visually negligible
bit plane	-	Binary layer formed by one bit position across all pixels
redundancy	-	Carrier parts that can be modified without obvious artifacts
capacity / payload	embedding capacity	Maximum secret bits an image can carry
relative payload	-	Embedded bits divided by available positions

embedding efficiency	alpha	Average secret bits per modification, alpha		
matrix embedding	-	Group coding that embeds p bits while changing at most one position		
Hamming code	binary Hamming code	Linear error-locating code with block [n,k,3]		
parity-check matrix	H matrix	p x n matrix with distinct columns used to compute syndromes		
syndrome	-	s = H*x (mod 2); a summary of the block state		
GF(2)	Galois field of two elements	Field containing only 0/1 with XOR arithmetic		
shrinkage	-	Coefficient magnitude decrease lands on zero, ruining the block		
wet paper coding	WPC	Embedding on dry positions while wet positions stay untouched		
dry / wet positions	-	Modifiable positions / untouchable positions		
F5 / nsF5	-	JPEG matrix-embedding algorithm; nsF5 removes shrinkage with wet paper		
magnitude decrease	coefficient shrink	Reducing \	coefficient\	by 1 to flip LSB parity
SHA-256	-	256-bit cryptographic hash used for image keying		

keying	-	Using password/content keys to decide hiding positions
deterministic permutation	-	Shuffle where the same seed always gives the same order
splitmix64	-	64-bit PRNG used by the project
Fisher-Yates	-	Standard uniform shuffle algorithm
self-synchronization	-	Decoder finds the payload without external parameters
tamper perception	-	Detecting that an image was modified
blind steganalysis	-	Statistical detection without the original cover
chi-square test	Westfeld test	Checks whether gray-level pairs were balanced
p-value	-	Probability that observations fit the assumption; here high p is suspicious
RS analysis	-	Counts regular/singular groups under +/- masks
regular / singular	R / S groups	Groups whose discriminant rises / falls after flipping
difference entropy	-	Shannon entropy of adjacent differences; measures texture randomness
Shannon entropy	-	Information/uncertainty measure in bits

supervised learning	-	Training a model on labeled samples
feature vector	-	Numeric description of a sample (11-D v1 or 143-D v2 here)
label	-	Ground truth (0 = clean, 1 = stego)
binary classification	-	Predicting membership of one of two classes
logistic regression	-	Linear combination + sigmoid; an interpretable classifier
sigmoid	-	S-shaped function mapping any real number to 0-1
cross-entropy loss	-	Standard classification loss
gradient descent	-	Iteratively moving parameters downhill in loss
overfitting	-	Memorizing training data at the cost of generalization
cross-validation	-	Rotating validation folds to estimate generalization
data leakage	-	Validation information entering training, inflating metrics
GroupKFold	-	Grouped cross-validation (same photo's samples stay together)
confusion matrix	-	Four-cell counts of truth vs prediction

precision / recall	-	Share of flagged that are real / share of real that are caught
ROC / AUC	-	Detection-vs-FP curve across thresholds and its area
Youden's J	-	Threshold maximizing TPR - FPR
false positive / negative	FP / FN	Clean flagged as stego / stego missed
SRM	Spatial Rich Model	High-pass residual filter family that highlights embedding noise
residual map	-	Noise residual after high-pass filtering
LightGBM / LGB	-	Gradient-boosting implementation used by the v1.4 models
feature group	-	A batch of features from one source (BASE/SRM/PREFIX/LSB-PREFIX)
out-of-distribution	OOD	Inputs unlike the training distribution (e.g., real JPEG clean)
BOSSbase	BOSSbase 1.01	Standard benchmark: 10,000 512x512 grayscale PGM images
PGM	-	Lossless grayscale image format (BOSSbase carrier format)
stacking	meta-learner	Ensemble where a second model combines base-model predictions

memmap	memory-mapped file	Large arrays written/read in slices to avoid OOM
calibration set	-	Separate subset used only for threshold selection
grid tuning	-	Searching a hyperparameter grid and comparing models
dual-version model	-	143d robust model + 53d interpretable model deployed together
ctypes	-	Python binding mechanism for C/C++ DLLs
DLL	dynamic-link library	Windows shared library
CUDA / batching	-	GPU parallelism / processing many samples at once
consistency check	-	Bit-level comparison between C++/GPU and Python outputs

Appendix B - Command Cheat Sheet

Run from the project root F:\Steganography.

Purpose	**Command**
Install core dependencies	pip install numpy pillow
Install everything (ML/plot)	pip install -e .
Core algorithm self-test	python src\test_core.py
Steganalysis self-test	python src\test_steg.py
False-positive regression	python src\test_false_positive.py
GUI smoke test	python src\test_gui.py
End-to-end verification	python src\run_e2e.py
Launch the GUI	python src\gui.py
Code-family/efficiency plot	python -c "import sys; sys.path.insert(0,'src'); from efficiency import plot_code_family_and_efficiency; print(plot_code_family_and_efficiency())"
Generate v1 dataset	python src\make_dataset.py data\campus_jpg –out campus
Generate v2 dataset (12 variants / 143-D)	python src\make_dataset.py –out campus_v2 –feature-set v2 –variants all
Generate SRM-enhanced dataset (same source)	python src\make_dataset.py data\campus_jpg –out campus_srm –preprocess srm -j 16
Train/evaluate a dataset	$env:DS_FILES = "dataset.csv"; python src\train_model.py
Train on v2 + JPEG clean	$env:DS_FILES = "dataset_campus_v2_jpeg.csv"; python src\train_model.py
Single-image ML prediction	python -c "import sys; sys.path.insert(0,'src'); from ml_predict import get_predictor; import numpy as np; from PIL import Image; print(get_predictor().predict(np.asarray(Image.open('img/cover.png'

Switch to 53d interpretable model	python -c ”import sys; sys.path.insert(0,’src’); from ml_predict import MLPredictor; import numpy as np; from PIL import Image; print(MLPredictor(model_path=’models/stego_classifier_v2_jpeg_ clip_outliers=False).predict(np.asarray(Image.open(’img/cover.png’)
C++/Python consistency check	python -c ”import sys; sys.path.insert(0,’src’); import cppembed; cppembed.selfcheck()”
GPU dataset generation	python gpu\make_imageset.py
GPU feature self-check	python gpu\featurize_gpu.py
GPU v2 feature self-check	python gpu\featurize_v2_gpu.py
GPU training	python gpu\train_ml_gpu.py
GPU single-image detection	python gpu\predict_gpu.py img\cover.png
BOSSbase multi-source (GPU)	python gpu\make_imageset.py data\BOSSbase_1.01 –out bossbase, then python gpu\train_ml_gpu.py
Build packages	python -m build
Install wheel	pip install dist\nsf5stego-1.4.0-py3-none-any.whl

Appendix C - 12-Week Checklist

Week	Topic	Required action	Date
1	Images & binary	Run the bit operations; state what LSB means	
2	Python environment	pip install; run run_e2e.py; save one grayscale image	
3	LSB & blind analysis	Embed -> analyze experiment; read test_steg.py output	
4	Chi-square / RS	Explain Gn collapse to a friend; run test_false_positive.py	
5	Matrix embedding / F5	Work one p=3 example; verify with matrix_demo	
6	nsF5 / wet paper	Read the wet-paper section of ns5_core.py; run its test	
7	Hash keying	Wrong-password and one-pixel experiments; write the keying boundary	
8	ML foundations	Run v1 training; explain GroupKFold and AUC	
9	ML steganalysis	Compare 11-D / 143-D / 53-D predictions; explain SRM and dual models	
10	Engineering	Run cppembed.selfcheck and featurize_v2_gpu; read SRM/multi-source pipeline	
11	Capstone	Choose a direction and finish 70% of code/experiments	

12	Presentation	Finish report and demo; answer three questions from 10.5 unaided

Appendix D - Concept-to-Code Map

Concept	File	Read first
Message encoding / length header	src/ns5_core.py	encode_string / decode_string
Image hash / seeds	src/ns5_core.py	derive_seed / get_image_hash
Deterministic permutation	src/ns5_core.py	permute_index
Hamming code / syndrome	src/ns5_core.py	build_hamming / syndrome
Matrix embedding	src/ns5_core.py	MatrixEmbedding._embed
Wet paper solver	src/ns5_core.py	solve_wet_paper / gauss_solve_GF2
Pixel-domain nsF5	src/ns5_core.py	nsF5Pixel._embed
High-level embed/extract	src/ns5_core.py	embed_string / extract_string
Chi-square statistics	src/steganalysis.py	chi2_stats / chi2_sf
RS statistics	src/steganalysis.py	rs_metrics
Combined verdict	src/steganalysis.py	analyze
v1 feature order	src/fsfeatures.py / ml_predict.py	FEAT_KEYS / V1_FEAT_KEYS
v2 143-D features	src/featurize_v2.py	ALL_FEATURE_NAMES / featurize_v2()
SRM preprocessing	src/srm_filter.py	srm_residuals_np / preprocess_batch_torch
Training pipeline	src/train_model.py	main()
Dataset generation	src/make_dataset.py	main()
C++ embedding wrapper	src/cppembed.py	embed_string / selfcheck
Dual-model inference	src/ml_predict.py	MLPredictor(model_path=..., clip_outliers=...)
GPU v1 features	gpu/featurize_gpu.py	batch operators / check_vs_cpu
GPU v2 features	gpu/featurize_v2_gpu.py	extract_features_v2_gpu() / self-check
GUI callbacks	src/gui.py	button events -> core functions

Appendix E - Recommended Resources

Books

- Zhou Zhihua, Machine Learning (in Chinese) - especially Chapter 2 on model evaluation and selection;
- Hang Li, Statistical Learning Methods (in Chinese) - classic derivations of logistic regression and perceptrons;
- Gonzalez & Woods, Digital Image Processing - bit planes, histograms, and frequency-domain fundamentals (early experiments used its images; v1.4 data sources switched to campus photos and BOSSbase);
- Any Chinese textbook on information hiding and digital watermarking as broad reading.

Core Papers (in reading order)

- Westfeld, F5 - A Steganographic Algorithm, 2001 - matrix embedding and F5;
- Westfeld & Pfitzmann, Attacks on Steganographic Systems, 1999 - chi-square test;
- Fridrich, Goljan & Du, Reliable Detection of LSB Steganography in Color and Grayscale Images, 2001 - RS analysis;
- Fridrich, Goljan, Lisonek & Soukal, Writing on Wet Paper, 2005 - wet paper coding;
- Pevny, Filler & Bas, Using High-Dimensional Image Statistics to Perform Unsigned Steganalysis - modern feature-based steganalysis (SPAM) entry point.

Online

- The project README and the thesis draft thesis/thesis1.pdf (SRM / 143-D / 53-D experiments are in the experimental chapter); GitHub: Yukinoshita-lin/nsf5-steganography;
- scikit-learn docs: LogisticRegression, GroupKFold, roc_curve;

- PyTorch tutorials (needed for the GPU pipeline);
- GitHub Actions documentation for CI/release.

Try it

The handbook ends here. Return to the checklist in 0.4 and grade yourself item by item. If you can tick all of them, give yourself a certificate: nsF5 Steganography - From Zero to Understanding.

Appendix F - A 6-12 Month In-Depth Self-Study Roadmap

Try it

Section 0.3 in the intro is the **12-week "get started"** compressed route; this section is the **6-12 month "understand and produce results"** deep route. It is for people who want to truly absorb digital image processing, information hiding, and machine learning, and do research/engineering. **Both routes use the same content; only the time allocation differs.**

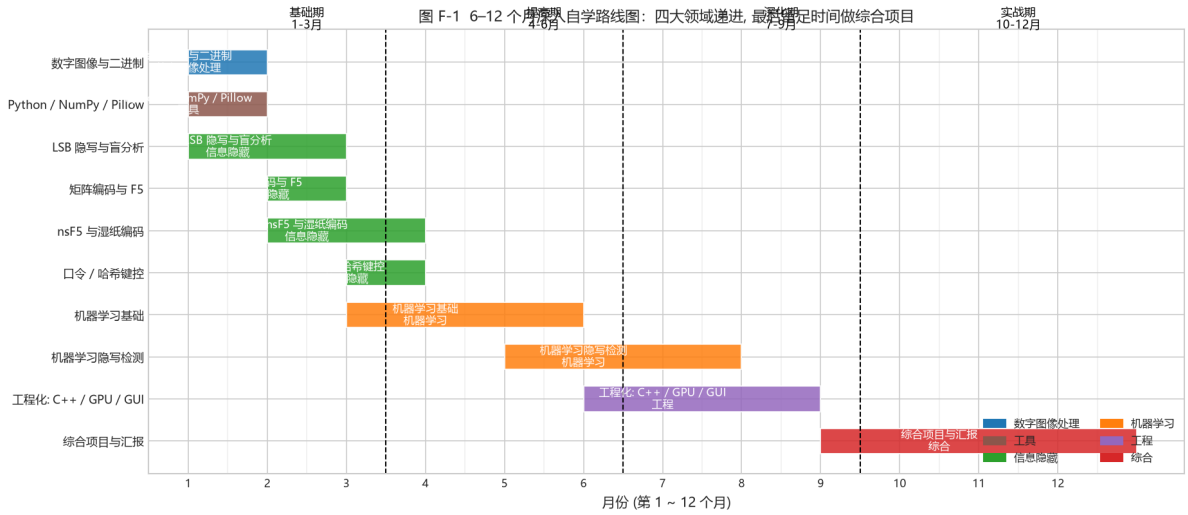

图 34: Fig. F-1 roadmap

Fig. F-1 (6-12 month Gantt: x-axis = month, y-axis = topic, color = field; dashed lines split it into Foundation (1-3) / Intermediate (4-6) / Advanced (7-9) / Project (10-12))

F.1 Four Phases: Goals and Deliverables of Each

Phase	Months	Goal	Main content	What you can do at the end

Foundation	1-3	Solidify the ”groundwork” of all three fields	ch01 digital images & binary, ch02 Python toolchain, ch03 LSB hiding & blind analysis, ch04 matrix embedding & F5, ch05 nsF5 wet paper, ch06 hash keying	Independently ”hide a sentence -> catch it with chi-square/RS”; hand-compute a p=3 Hamming embedding; explain wet-paper decoding
Intermediate	4-6	Enter machine learning; build the ”features->model->evaluation” frame	ch07 ML foundations, ch08 ML steganalysis (11-D -> 143-D -> dual models)	Explain overfitting/data leakage/GroupKFold/AUC; read and reproduce the v1 11-D training pipeline
Advanced	7-9	Absorb the engineering and the ”why designed this way”	ch08.8 SRM and 143d/53d strategy, ch09 engineering (C++ / GPU / GUI / multi-source data)	Read SRM filtering and 143-D features; understand same-source vs cross-source differences; run GPU batch features and self-checks
Project	10-12	Do one complete work of ”your own”	ch10 capstone, ch11 report; plus one research or engineering track	Complete a reproducible improvement experiment with an **honest** result analysis; present without notes

Tip

”Foundation” is almost identical to the 12-week route, just spending more time thinking through every **why** (especially Chapter 3’s statistical detection and Chapter 5’s wet paper). The real difference is in ”Advanced” and ”Project”.

F.2 Four Tracks: Taking ”Broad, Complete, Practical” through

To produce results, do not spread evenly. Go deep on one track; keep the rest at ”understands it” level:

Track	Focus	Deep-dive references
Algorithm	nsF5/F5, matrix embedding, wet paper, syndrome	ch04-05, project `ns5_core.py`; extend to JPEG domain & DCT coefficients
Detection / ML	feature engineering, SRM, 143d/53d, OOD	ch08, `featurize_v2.py`, `train_model.py`; extend to modern high-dim features & deep steganalysis
Engineering	C++ DLL, GPU batch, GUI, multi-source datasets, CI	ch09, `cppembed.py`, `gpu/*`, `.github/workflows`; extend to deployment & robustness
Research	reproduce papers -> find a gap -> improve -> honest evaluation	papers (appE) and `thesis/`; emphasize the split/calibration/test discipline (ch07)

F.3 Optional "Go Further" Topics (pick 1-2 by track)

- **JPEG-domain hiding**: move ch04-05 to quantized DCT coefficients (the project's `yccstego` extension is this direction); feel "spatial vs transform domain" difference;
- **Deep steganalysis**: under small-sample constraints, use explainable features first to verify the signal, then cautiously introduce deep models (the ch08.1 lesson);
- **Domain adaptation / cross-source robustness**: SRM gains on same-source but loses on cross-source (ch08.8.1); study how adaptation makes features stable across cameras/compression;
- **Adversarial robustness and false-positive control**: study "loose vs strict" thresholds and Youden / low-FP trade-offs (ch07), and the physical ceiling of undetectable embedding;
- **Explainability**: 143d robust vs 53d interpretable (ch08.8.4); study which features truly contribute the gain and how to explain per-dimension.

Back to the code

Land any of the above on code: first reproduce the baseline, then make a small change, and use `GroupKFold` for an honest comparison. **"Changed A, result got better" is not enough; prove it truly got better using the train/calibration/test three-layer structure + grouped split.** (ch07, ch08 stress this repeatedly.)

F.4 Relation to the 12-Week Route (0.3)

- **Want to get started quickly**: follow 0.3; in 12 weeks, tick off Appendix C;

- **Want to go deep and produce results**: follow this section; over 6-12 months, **slow down, deepen, and connect** the 0.3 weekly tasks, and add the parts of the 0.5 project map that are only truly needed for research (SRM, multi-source, GPU, dual models, OOD validation) in the "Advanced/Project" phases.
- **Common bottom line for both**: never skip the "hands-on experiments", and never skip "honest evaluation". The most important thing this handbook wants to teach is not a single algorithm but **"how to tell whether an experimental conclusion is trustworthy"**.

Think about it

Which track do you plan to take? How long is your cycle? (12 weeks vs 6-12 months means you spend time on "width" or "depth".) Write it down as your personal goal for this book.